NSW Property Market Analytics Report

End-to-End Business Analysis: Data Cleaning in Python | Containerized SQL in Azure SQL Edge | Executive Dashboard in Tableau

---

1. Project Overview & Business Context

This project analyzes real estate sales transactions across New South Wales (NSW) using historical transaction data. The primary objective is to evaluate property investment opportunities by establishing data-backed frameworks for capital growth, price per square meter, property type performance, and locality-level opportunity scoring.

The end-to-end analytical pipeline processes over 1.5 million raw sales records through a Python ETL script, stores the clean relational data in an Azure SQL Edge database running on Docker, executes structured exploratory T-SQL analysis, and delivers interactive executive insights through a Tableau dashboard.

2. Dataset Summary

• Raw Dataset: ~1.508 Million Rows
• Processed Dataset: 1,121,519 Cleaned Rows | 14 Standardized Attributes
• Core Attributes: property_id, unit_number, house_number, street_name, locality, postcode, area_m2, contract_date, settlement_date, purchase_price, nature_of_property, primary_purpose, dealing_number, council_name
• Coverage: NSW Statewide Property Transactions (https://nswpropertysalesdata.com/)

```
[268]: df.columns
```

```
[268]: Index(['District code', 'Property ID', 'Valuation number', 'Sale counter',
              'Download date / time', 'Property name', 'Property unit number',
              'Property house number', 'Property street name', 'Property locality',
              'Property post code', 'Area (m2)', 'Area (source)',
              'Area type (source)', 'Dimensions', 'Contract date', 'Settlement date',
              'Purchase price', 'Zoning', 'Nature of property',
              'Nature of property (description)', 'Primary purpose',
              'Primary purpose (normalised)', 'Strata lot number', 'Component code',
              'Sale code', '% Interest of sale', 'Dealing number', 'Parcels in sale',
              'Property legal description', 'Part interest flag',
              'Non-market price flag', 'Date error flag'],
             dtype='object')
```

3. Exploratory Data Cleaning using Python (Pandas)

Data cleaning and transformation were performed using Python (Pandas) in Jupyter Notebook:
1. Schema Rationalization: Dropped 14 non-essential columns (Zoning, Valuation number, Sale counter).

[269]:
```
# Drop Sale counter since its not needed for our business case
# Contract Dates might have errors in it so use Dealing number to identify Sale contract
# Purchase price is not always the whole property price and might include properties that have stakes in the properties
# so we will eliminate all the properties that have part interest flag since we only want properties that get us 100% ownership
# There might be rows with multiple same price values caused due to Parcels in sale column so we will try to get only 1 row from them
# in order to maintain consistency, we can do df['Parcels in sale']=1 or group by Dealing number
# drop zoning since we wont need that for our use case
# drop the rows where Non-market price flag is set since they are all family transfers or having sale price less than 1000$
# which is not true market value

# Columns to be dropped that wont be needed for our use case

# Sale counter, Download date/time, Property name, Area (source), Area type (source), Dimensions,
# Zoning, Nature of property, Primary purpose, Component Code, Sale code, Property legal description,  Strata lot number

df = df.drop(['Sale counter', 'Download date / time', 'Property name', 'Area (source)', 'Area type (source)',
                    'Dimensions', 'Zoning', 'Nature of property', 'Primary purpose', 'Component code', 'Valuation number', 'Sale code', 'Str
```

[270]:
```
df.head(3)
```

[270]:

	District code	Property ID	Property unit number	Property house number	Property street name	Property locality	Property post code	Area (m2)	Contract date	Settlement date	Purchase price	Nature of property (description)	Primary purpose (normalised)	% Interest of sale	Dealin numb
0	4	109747.0	NaN	5	Columbia Cl	Woodrising	2284.0	800.0	2018-12-04	2018-12-28	437000	Residence	Residence	NaN	AN9732
1	4	110548.0	NaN	23	Southern Cross Dr	Woodrising	2284.0	719.3	2018-09-27	2018-12-11	438000	Residence	Residence	NaN	AN9679

2. Column Name Normalization: Standardized all column headers to lowercase snake_case.

[271]:
```
Index(['District code', 'Property ID', 'Property unit number',
       'Property house number', 'Property street name', 'Property locality',
       'Property post code', 'Area (m2)', 'Contract date', 'Settlement date',
       'Purchase price', 'Nature of property (description)',
       'Primary purpose (normalised)', '% Interest of sale', 'Dealing number',
       'Parcels in sale', 'Part interest flag', 'Non-market price flag',
       'Date error flag'],
      dtype='object')
```

[272]:
```
# Clean and transform the column names
df.columns = (df.columns
              .str.replace('%', 'percent', regex=False)        # Convert % to text
              .str.replace(r'[^a-zA-Z0-9\s]', '', regex=True)  # Remove brackets and symbols
              .str.replace(r'\s+', '_', regex=True)            # Change spaces to underscores
              .str.lower()                                     # Convert to lowercase
              .str.strip('_'))                                 # Remove any trailing underscores
```

[273]:
```
df.columns
```

[273]:
```
Index(['district_code', 'property_id', 'property_unit_number',
       'property_house_number', 'property_street_name', 'property_locality',
       'property_post_code', 'area_m2', 'contract_date', 'settlement_date',
       'purchase_price', 'nature_of_property_description',
       'primary_purpose_normalised', 'percent_interest_of_sale',
       'dealing_number', 'parcels_in_sale', 'part_interest_flag',
       'nonmarket_price_flag', 'date_error_flag'],
      dtype='object')
```

3. Parcel & Ownership Filtering: Filtered for single-parcel transactions (parcels_in_sale == 1) and full interest sales (part_interest_flag == False).

[284]:
```
# Converting these objects to their respective data types so we can get rid of the duplicates and unneeded data for our use case
# Convert parcels_in_sale to numbers, turning invalid text into NaN (Missing Values)
df['parcels_in_sale'] = pd.to_numeric(df['parcels_in_sale'], errors='coerce')

# Convert the part_interest_flag, nonmarket_price_flag and date_error_flag to boolean
df = df.astype({'part_interest_flag': 'bool',
       'nonmarket_price_flag':'bool', 'date_error_flag':'bool'})
```

[285]:
```
# Get only the rows that have one parcel since multiple parcels would show us multiple instances of the same price for the same sale
df = df[df['parcels_in_sale']==1]
```

[286]:
```
df.columns
```

[286]:
```
Index(['district_code', 'property_id', 'property_unit_number',
       'property_house_number', 'property_street_name', 'property_locality',
       'property_post_code', 'area_m2', 'contract_date', 'settlement_date',
       'purchase_price', 'nature_of_property_description',
       'primary_purpose_normalised', 'percent_interest_of_sale',
       'dealing_number', 'parcels_in_sale', 'part_interest_flag',
       'nonmarket_price_flag', 'date_error_flag'],
      dtype='object')
```

4. Non-Market Price & Shared Ownership Exclusion: Removed non-market transactions and transfers under 1,000 and also transactions that have multiple people owning the same property.

[287]:
```
# Also we will be eliminating the rows where the transfers have prices less that 1000$ since they might be family transfers
# and do not hold actual market value
df = df[df['nonmarket_price_flag'] == False]

# Here we will be getting rid of the properties that have shared ownership since we want 100% ownership for the property
df = df[df['part_interest_flag'] == False]
```

[288]:
```
# Drop these columns since they are not needed now
df = df.drop(columns = ['nonmarket_price_flag', 'part_interest_flag', 'parcels_in_sale', 'percent_interest_of_sale'])
```

[289]:
```
df.head()
```

[289]:

	district_code	property_id	property_unit_number	property_house_number	property_street_name	property_locality	property_post_code	area_m2	contract_
0	4	109747.0	NaN	5	Columbia Cl	Woodrising	2284.0	800.0	2018-1
1	4	110548.0	NaN	23	Southern Cross Dr	Woodrising	2284.0	719.3	2018-0
2	4	135349.0	NaN	11	George St	Highfields	2289.0	688.8	2018-1
3	4	139881.0	NaN	57	Scenic Dr	Caves Beach	2281.0	689.2	2018-
6	4	161585.0	NaN	5	Haynes Ave	Eleebana	2282.0	619.7	2018-1

x

5. Missing Postcode Imputation: Imputed null postcodes using locality lookup dictionary.

[295]:
```
# same locality should have the same postcode so we can map these into the data
df['property_post_code'].isnull().sum()
```

[295]:
```
np.int64(932)
```

[296]:
```
# 1. Create a clean mapping dictionary from valid rows
# Drops rows where postcode or locality is missing, then drops duplicate localities
postcode_map = (
    df.dropna(subset=["property_locality", "property_post_code"])
    .drop_duplicates(subset=["property_locality"])
    .set_index("property_locality")["property_post_code"]
    .to_dict()
)

# 2. Fill the missing postcodes using the map
df["property_post_code"] = df["property_post_code"].fillna(df["property_locality"].map(postcode_map))

# 3. Optional: Convert postcode to float64
df["property_post_code"] = df["property_post_code"].astype("float64")
```

[297]:
```
df['property_post_code'].isnull().sum()
# Successfully dropped the number of missing postcodes from 932 to 15
```

[297]:
```
np.int64(15)
```

[298]:
```
df.loc[df['property_post_code'].isna(),:]
# These properties have no name or any information related to the property so lets eliminate data like this
```

[298]:

	district_code	property_id	property_unit_number	property_house_number	property_street_name	property_locality	property_post_code	area_m2	co
1282295	210	6005920.0	NaN	NaN	NaN	NaN	NaN	0.0	
1282296	210	6005920.0	NaN	NaN	NaN	NaN	NaN	0.0	
1282297	210	6005920.0	NaN	NaN	NaN	NaN	NaN	0.0	

6. Dropping rows if the postcode, locality, street name or house number is null and converting all of those columns to string.

[299]:
```
# would drop the columns if these identifiers are not present
# List of columns to check
target_cols = [
    "property_post_code",
    "property_locality",
    "property_street_name",
    "property_house_number"
]

# Drop rows only if these columns are NaN
for col in target_cols:
    df = df.dropna(subset=[col])
```

[300]:
```
df.loc[df['property_post_code'].isna(),:]
```

[300]:

	district_code	property_id	property_unit_number	property_house_number	property_street_name	property_locality	property_post_code	area_m2	contract_d

[301]:
```
# Changing all the property location variables to string
df['property_unit_number'] = df['property_unit_number'].astype('string')
df['property_house_number'] = df['property_house_number'].astype('string')
df['property_street_name'] = df['property_street_name'].astype('string')
df['property_locality'] = df['property_locality'].astype('string')
```

[302]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1416598 entries, 0 to 1508051
```

7. Council District Mapping: Mapped 3-digit district codes to official LGA Council names.

[305]:
```
# We have the Council code but dont have the Council that it maps to so we shall do the mapping ourselves

council_code_map = {
    "001": "Cessnock", "118": "Blayney", "240": "Gilgandra",
    "002": "Dungog", "123": "Oberon", "243": "Hay",
    "004": "Lake Macquarie", "124": "Orange", "244": "Lachlan",
    "005": "Maitland", "137": "Burwood", "247": "Narrabri",
    "007": "Muswellbrook", "139": "Canada Bay", "250": "Tenterfield",
    "008": "Newcastle", "143": "Strathfield", "251": "Narromine",
    "010": "Port Stephens", "144": "Sutherland", "252": "Walcha",
    "012": "Singleton", "148": "Ballina", "253": "Walgett",
    "018": "Bega Valley", "149": "Bellingen", "254": "Warren",
    "042": "Cowra", "150": "Byron", "255": "Wentworth",
    "043": "Weddin", "151": "Richmond Valley", "257": "Armidale Regional",
    "050": "Albury", "152": "Coffs Harbour", "258": "Canterbury-Bankstown",
    "051": "Berrigan", "157": "Kempsey", "259": "Central Coast",
    "052": "Carrathool", "158": "Kyogle", "260": "City of Parramatta",
    "054": "Coolamon", "159": "Lismore", "261": "Cumberland",
    "061": "Junee", "164": "Nambucca", "262": "Edward River",
    "065": "Leeton", "171": "Tweed", "263": "Federation",
    "066": "Lockhart", "187": "Gunnedah", "264": "Georges River",
    "070": "Narrandera", "188": "Inverell", "265": "Cootamundra-Gundagai Regional",
    "074": "Griffith", "192": "Moree Plains", "266": "Hilltops",
    "081": "The Hills Shire", "199": "Uralla", "267": "Inner West",
    "082": "Hornsby", "207": "Randwick", "268": "Mid-Coast",
    "083": "Hunters Hill", "209": "Waverley", "269": "Murray River",
    "084": "Ku-ring-gai", "210": "Woollahra", "270": "Murrumbidgee",
    "085": "Lane Cove", "214": "Blacktown", "271": "Northern Beaches",
    "087": "Mosman", "216": "Blue Mountains", "272": "Queanbeyan-Palerang Regional",
    "088": "North Sydney", "217": "Camden", "273": "Snowy Monaro Regional",
    "089": "North Sydney", "218": "Campbelltown", "274": "Snowy Valleys", # Note: 088 / 089 are North Sydney
    "090": "Ryde", "219": "Hawkesbury", "275": "Dubbo Regional",
    "092": "Willoughby", "220": "Fairfield", "276": "Bayside",
    "097": "Eurobodalla", "222": "Lithgow", "300": "Gwydir",
```

```
    "235": "Central Darling", "236": "Cobar", "538": "Temora",
    "238": "Coonamble", "239": "Bourke", "560": "Greater Hume",
    "575": "Wagga Wagga", "608": "Bathurst Regional", "620": "Mid Western Regional",
    "656": "Port Macquarie-Hastings", "666": "Tamworth Regional", "708": "City of Sydney",
    "902": "Unincorporated Area", "903": "Unincorporated Sydney Harbour"
}

# Map the codes to names into a new column
df["council_name"] = df["district_code"].map(council_code_map)
```

[306]:
```
df["council_name"].value_counts(dropna=False)
```

[306]:
```
council_name
Blacktown               68065
Central Coast           61862
City of Sydney          47794
City of Parramatta      45696
Canterbury-Bankstown    41106
                        ...
Bourke                    453
Balranald                 380
Central Darling           333
Brewarrina                191
Unincorporated Area        49
Name: count, Length: 130, dtype: int64
```

[307]:
```
df["council_name"].isna().sum()
# Some codes dont have a council that we can map to
```

[307]:
```
np.int64(1776)
```

[308]:
```
df["district_code"].isna().sum()
```

[308]:
```
np.int64(0)
```

[310]:
```
# Filling unknown district codes with Unknown as the label also getting rid of the disctrict code column since its redundant
df['council_name']= df['council_name'].fillna("Unknown").astype('string')
df = df.drop(columns=['district_code'])
```

[311]:
```
df['council_name'].isna().sum()
```

[311]:
```
np.int64(0)
```

[312]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1416598 entries, 0 to 1508051
Data columns (total 15 columns):
 #   Column                          Non-Null Count    Dtype
---  ------                          --------------    -----
 0   property_id                     1416598 non-null  float64
 1   property_unit_number            453318 non-null   string
 2   property_house_number           1416598 non-null  string
 3   property_street_name            1416598 non-null  string
 4   property_locality               1416598 non-null  string
 5   property_post_code              1416598 non-null  float64
 6   area_m2                         1128158 non-null  float64
 7   contract_date                   1416290 non-null  object
 8   settlement_date                 1416577 non-null  object
```

8. Date Error Correction: Swapped contract and settlement dates where contract_date > settlement_date.

[315]:
```
# checking the dates where the contract_date is appearing after settlement_date
df['incorrect_dates'] = df['contract_date']>df['settlement_date']
```

[316]:
```
#comparing it to the boolean already present in the table
(df['incorrect_dates'] == df['date_error_flag']).all()
```

[316]:
```
np.True_
```

[317]:
```
# We are going to assume that dates were misplaced and simply swap them for the ones that have incorrect_dates as True
# Create a boolean mask
mask = df['incorrect_dates']

# Use .loc with the mask to select rows, and swap the two columns
df.loc[mask, ['contract_date', 'settlement_date']] = df.loc[mask, ['settlement_date', 'contract_date']].values
```

[318]:
```
# Recheck if we have date issues
df['incorrect_dates'] = df['contract_date']>df['settlement_date']
```

[319]:
```
df['incorrect_dates'].all()==False
```

[319]:
```
np.True_
```

[320]:
```
# Now we can drop both the columns that were present to help the swap for dates
df = df.drop(columns=['incorrect_dates', 'date_error_flag'])
```

[321]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1416290 entries, 0 to 1508051
Data columns (total 14 columns):
 #   Column                          Non-Null Count    Dtype
```

9. Area Imputation & Outlier Removal: Imputed zero land areas via median $/m^2 estimates, keeping area_m2 > 10m^2 and price/m^2 >= 0.3, Also checked for outliers in both and decided to keep the values since they seem plausible.

[5416 rows x 16 columns]

[328]:
```
df['area_m2'].isna().sum()
```

[328]:
```
np.int64(290167)
```

[329]:
```
# 2920167 rows have no area_m2 which we will find using median price per m2 based of year and postcode
# which we will then divide it with purchase price where the area_m2 is na
# We will only keep area_m2 greater that 10 metre square for our dataset
df["price_per_m2"] = (
    df.loc[df["area_m2"]>10,"purchase_price"]/ df.loc[df["area_m2"]>10,"area_m2"]
)

median_price_per_m2 = (
    df.loc[df["area_m2"] > 10]  # <-- Only include areas higher than 10
    .groupby(["property_post_code", "settlement_year"])["price_per_m2"]
    .transform("median")
)

df["estimated_area_m2"] = (
    df["purchase_price"] / median_price_per_m2
```

495918	Tindarey	0.000006	1120	187970000.0	2835.0	2021	

4327 rows × 6 columns

[337]:
```
# Now we shall remove those rows

df = df.loc[~(df['price_per_m2'].astype(float)<0.3),:]
```

[336]:
```
# Now we need a to setup a price to m2 lower bound since if a property is extremely cheap then most likely its not a real transaction
# or even if it is we want properties that is atleast 0.3 dollars per metre square

df.loc[df['price_per_m2'].astype(float)<0.3, ['property_locality','price_per_m2','purchase_price', 'area_m2', 'property_post_code', 'settlem
    "price_per_m2",
    ascending=False
)
```

[336]:

	property_locality	price_per_m2	purchase_price	area_m2	property_post_code	settlement_year
1055365	Carlaminda	0.299925	2350000	7835300.0	2630.0	2024
290317	Mungindi	0.299760	3500000	11676000.0	2406.0	2020
267619	Pericoe	0.299568	48500	161900.0	2550.0	2020
27286	Myrtle Park	0.299555	606000	2023000.0	2713.0	2017
151178	Camira	0.299483	220000	734600.0	2469.0	2019
...	...	...	...	...	...	...

[325]:
```
# IQR for purchase price by postcode + year
# Getting rid of the outliers so the data is fairly consistent and reliable based on purchase price
group_cols = ["property_post_code", "settlement_year"]

q1 = df.groupby(group_cols)["purchase_price"].transform(
    "quantile", 0.25
)

q3 = df.groupby(group_cols)["purchase_price"].transform(
    "quantile", 0.75
)

iqr = q3 - q1
k = 3
lower_bound = q1 - k * iqr
upper_bound = q3 + k * iqr

# Flag outliers
df["purchase_price_outlier"] = (
    (df["purchase_price"] < lower_bound) |
    (df["purchase_price"] > upper_bound)
)
# Check the outliers just in case we are throwing out useful information
#df.loc[df["purchase_price_outlier"], ['purchase_price', 'area_m2', 'property_post_code', 'settlement_year']].sort_values(
#    "purchase_price",
#    ascending=False
#)

df.loc[df['purchase_price_outlier'], ['purchase_price', 'area_m2', 'settlement_date', 'property_locality']].sort_values(by="purchase_price")
```

[325]:

	purchase_price	area_m2	settlement_date	property_locality
1166922	1000	626.2	2024-12-02	Erskine Park
216883	1000	NaN	2023-08-29	Wentworth Point

326]:
```
# All the values seem legit so we wont be removing these rows
#df = df.loc[
#    ~df["purchase_price_outlier"], :
#    ]
```

327]:
```
print(df.loc[df['purchase_price_outlier']])
```

```
         property_id property_unit_number property_house_number  \
33         3697107.0                <NA>                1245 A
116        1036071.0                <NA>                    45
121        1048042.0                <NA>                    14
122        1048070.0                <NA>                    15
129        3258543.0                <NA>                     5
...              ...                 ...                   ...
1507944    2731551.0                <NA>                   103
1508038    6017060.0                 401                  2 A
1508044    6026252.0                   1                    29
```

```
# IQR for purchase price by postcode + year
# Getting rid of the outliers so the data is fairly consistent and reliable based on price per metre square
group_cols = ["property_post_code", "settlement_year"]
df['price_per_m2']=df['price_per_m2'].astype(float)
q1 = df.groupby(group_cols)["price_per_m2"].transform(
    "quantile", 0.25
)

q3 = df.groupby(group_cols)["price_per_m2"].transform(
    "quantile", 0.75
)

iqr = q3 - q1
k = 3
lower_bound = q1 - k * iqr
upper_bound = q3 + k * iqr

# Flag outliers
df["price_per_m2_outlier"] = (
    (df["price_per_m2"] < lower_bound) |
    (df["price_per_m2"] > upper_bound)
)
# Check the outliers just in case we are throwing out useful information
df.loc[df["price_per_m2_outlier"], ['property_locality','price_per_m2','purchase_price', 'area_m2', 'property_post_code', 'settlement_year']]
    "purchase_price",
    ascending=False
)


#df.loc[
#    df["price_per_m2_outlier"], ['area_m2', 'purchase_price', 'price_per_m2', 'property_street_name']
#]
# Most of these values seem plausible so we will keep them in the dataset
```

10. After removing all the unnecessary columns and changing their datatype we will export this dataset to a csv file and also send it to our AzureSQLEdge Server running on Docker using SQLAlchemy .

```
[350]: df['contract_date'].isnull().sum()
```

```
[350]: np.int64(0)
```

```
[353…
# Sending all this cleaned data to the AzureSqlEdge running on Docker
# Format and URL-encode the connection attributes
from sqlalchemy import create_engine, text

# Dialect changes to mssql+pymssql
# Syntax: mssql+pymssql://username:password@hostname:port/database
connection_url = "mssql+pymssql://SA:Ajith1234@localhost:1433/nsw_property_sales"

engine = create_engine(connection_url)

with engine.connect() as connection:
    result = connection.execute(text("SELECT @@VERSION;"))
    print("Connected successfully:", result.scalar())
    table_name="nsw_property"
    df.to_sql(table_name, engine, if_exists="replace", index=False)

print("Data successfully loaded in table ", table_name, " database nsw_property_sales")
```

```
Connected successfully: Microsoft Azure SQL Edge Developer (RTM) - 15.0.2000.1574 (ARM64)
        Jan 25 2023 10:36:08
        Copyright (C) 2019 Microsoft Corporation
        Linux (Ubuntu 18.04.6 LTS aarch64) <ARM64>
Data successfully loaded in table  nsw_property  database nsw_property_sales
```

```
[354]: df.to_csv("clean_nsw_property_sales.csv", index=False)
```

```
[355]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

4. Data Analysis using SQL (Data Analysis & Market Insights)

1. Market Overview

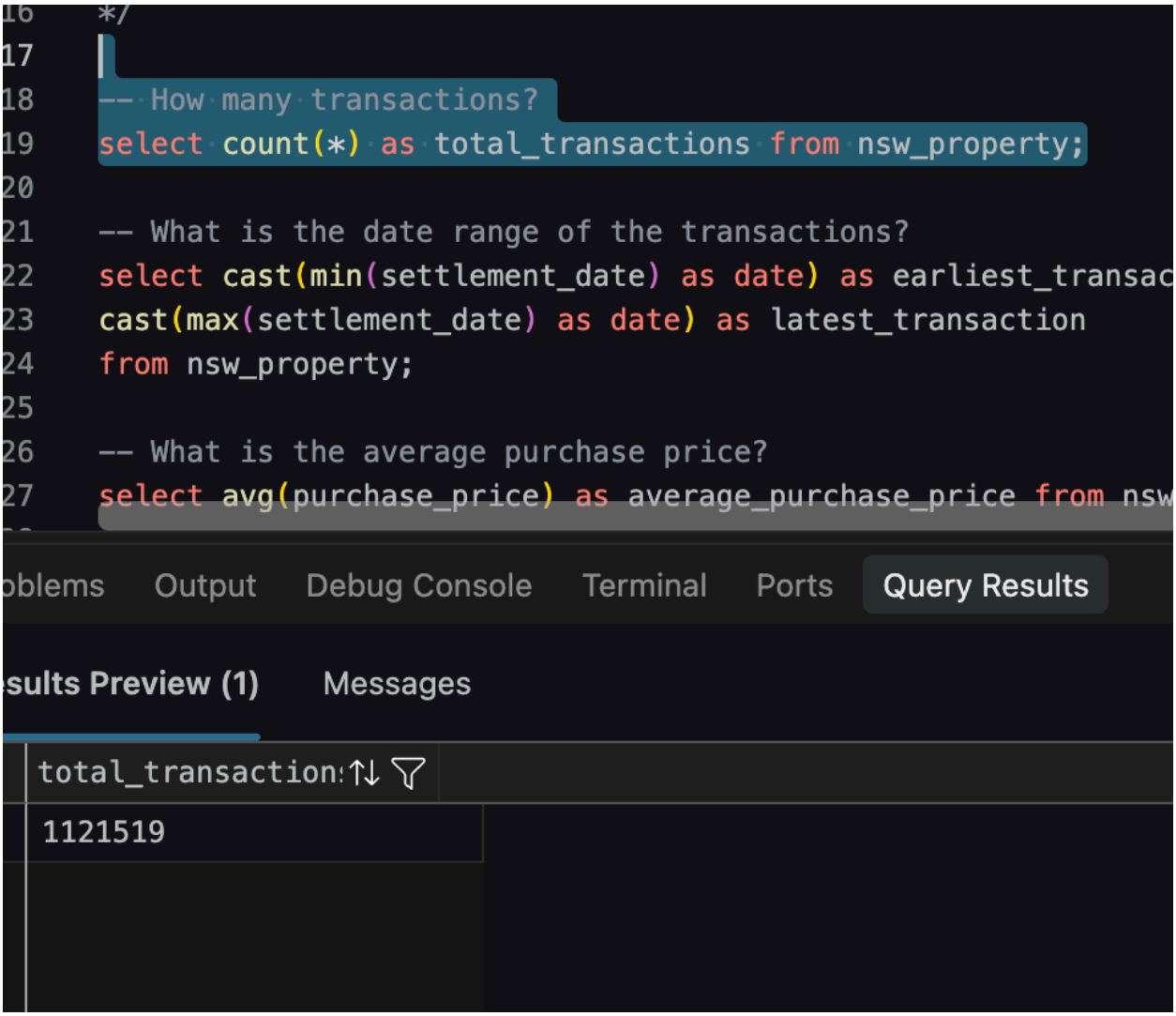
The first stage of the analysis focused on establishing a broad understanding of the NSW property market. Using the `nsw_property` dataset, SQL queries were used to determine the total number of transactions, the period covered by the dataset, the average purchase price, and changes in both property prices and transaction volumes over time.

Total Transactions

The first query calculated the total number of property transactions recorded in the dataset:

```
SELECT COUNT(*) AS total_transactions
FROM nsw_property;
```

This provides an initial indication of the size and scope of the dataset and establishes the number of individual transaction records available for analysis.

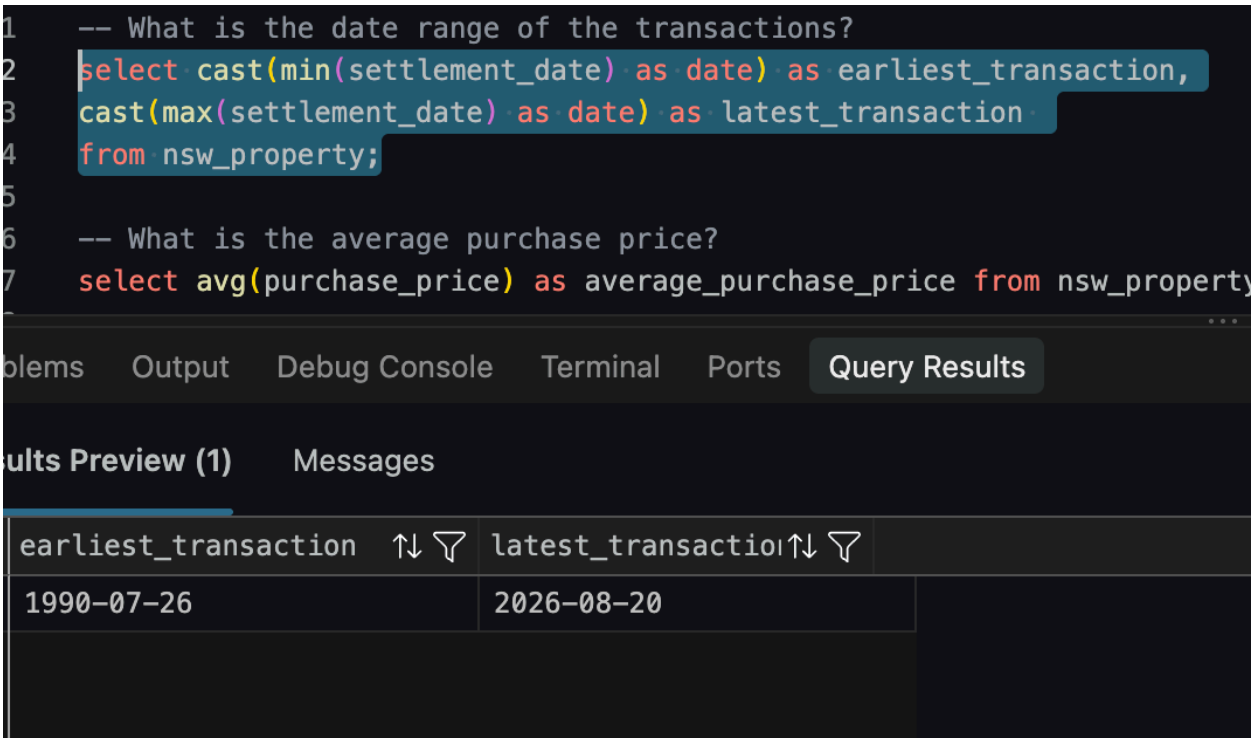


Transaction Date Range

The minimum and maximum settlement dates were then identified to determine the period covered by the dataset.

```
SELECT
    CAST(MIN(settlement_date) AS DATE) AS
earliest_transaction,
    CAST(MAX(settlement_date) AS DATE) AS
latest_transaction
FROM nsw_property;
```

Understanding the date range is important because it provides context for the subsequent time-series analysis. It also helps identify whether changes in property prices and transaction activity are being observed over a sufficiently long period.

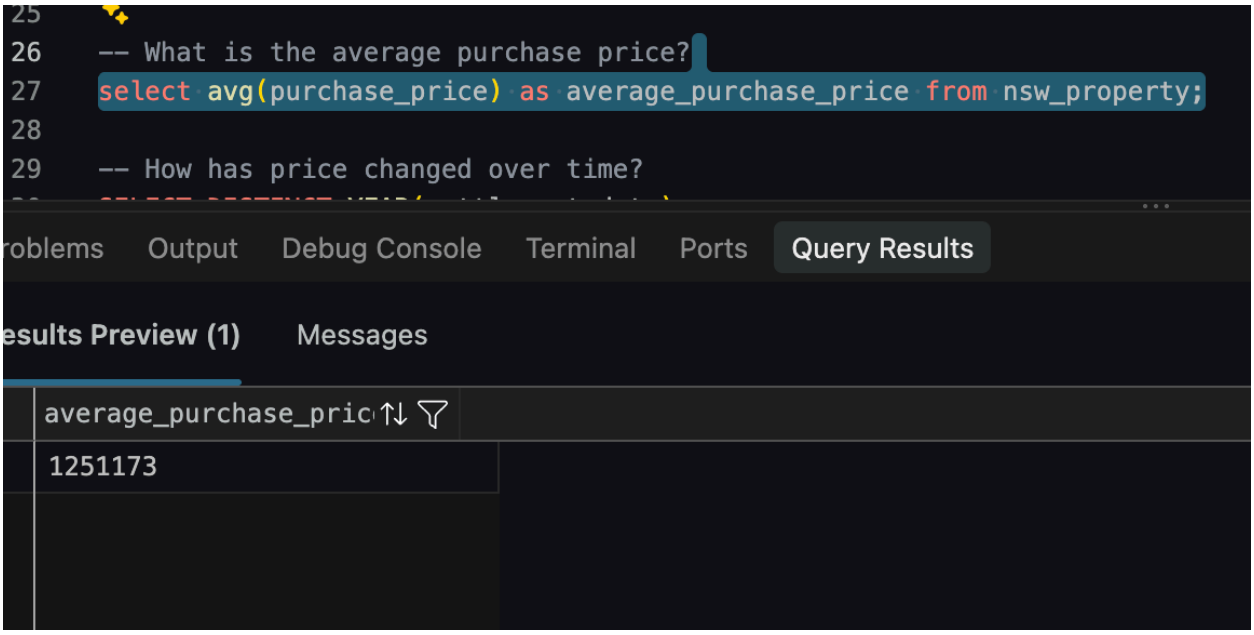


Average Purchase Price

The average purchase price was calculated to establish a baseline measure of property values across the dataset.

```
SELECT AVG(purchase_price) AS average_purchase_price
FROM nsw_property;
```

While the average provides a useful overall benchmark, it can be heavily influenced by very high-value transactions. For this reason, median prices were also examined when analysing changes over time and across different geographical areas.

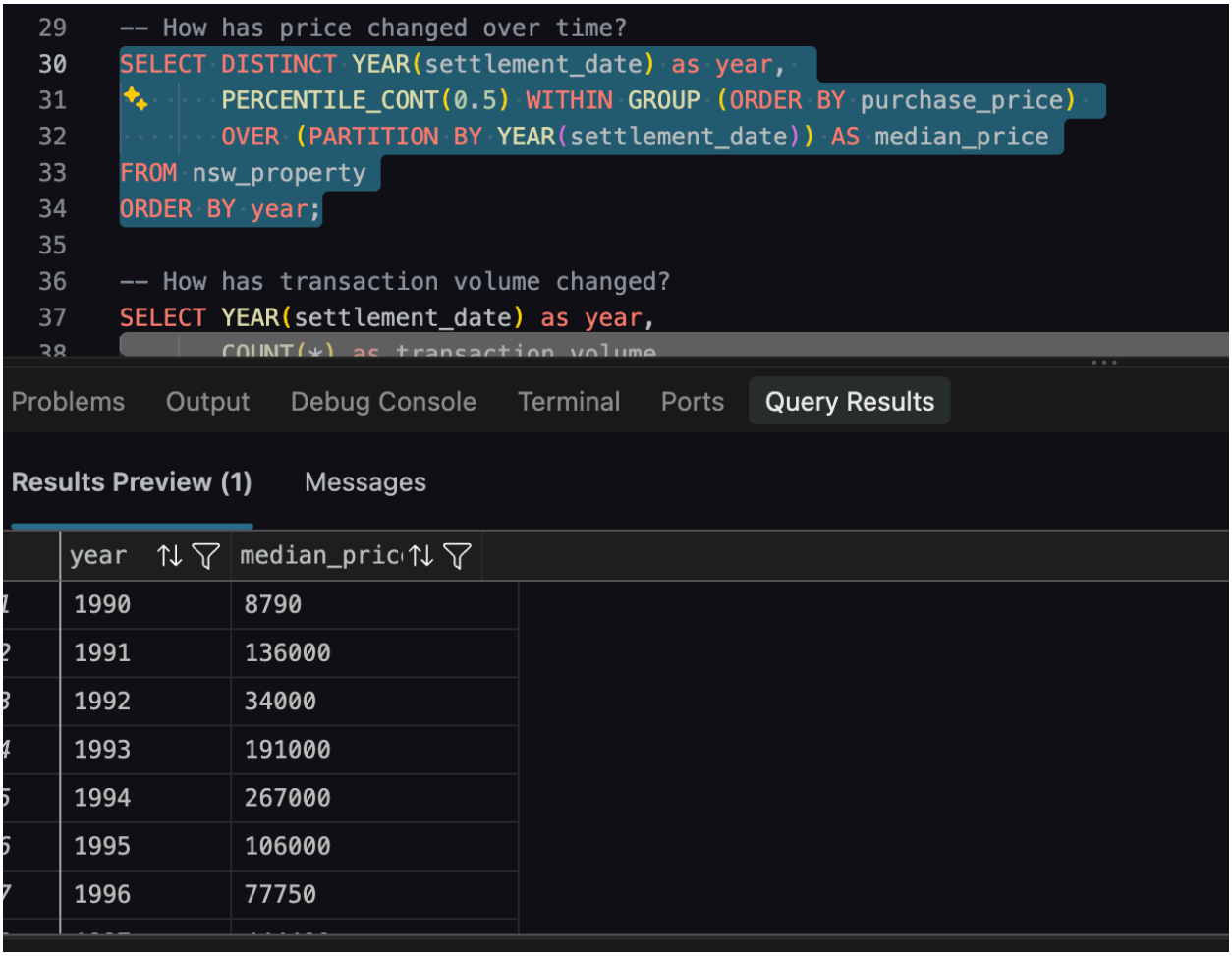


Changes in Property Prices Over Time

To understand how property prices changed over the period covered by the dataset, the median purchase price was calculated for each year.

```
SELECT DISTINCT
    YEAR(settlement_date) AS year,
    PERCENTILE_CONT(0.5)
        WITHIN GROUP (ORDER BY purchase_price)
        OVER (PARTITION BY YEAR(settlement_date)) AS
median_price
FROM nsw_property
ORDER BY year;
```

The use of the **median** rather than the average is particularly useful for property analysis because property prices can vary considerably. A small number of extremely expensive properties could significantly increase the average, whereas the median provides a more representative view of the typical transaction.



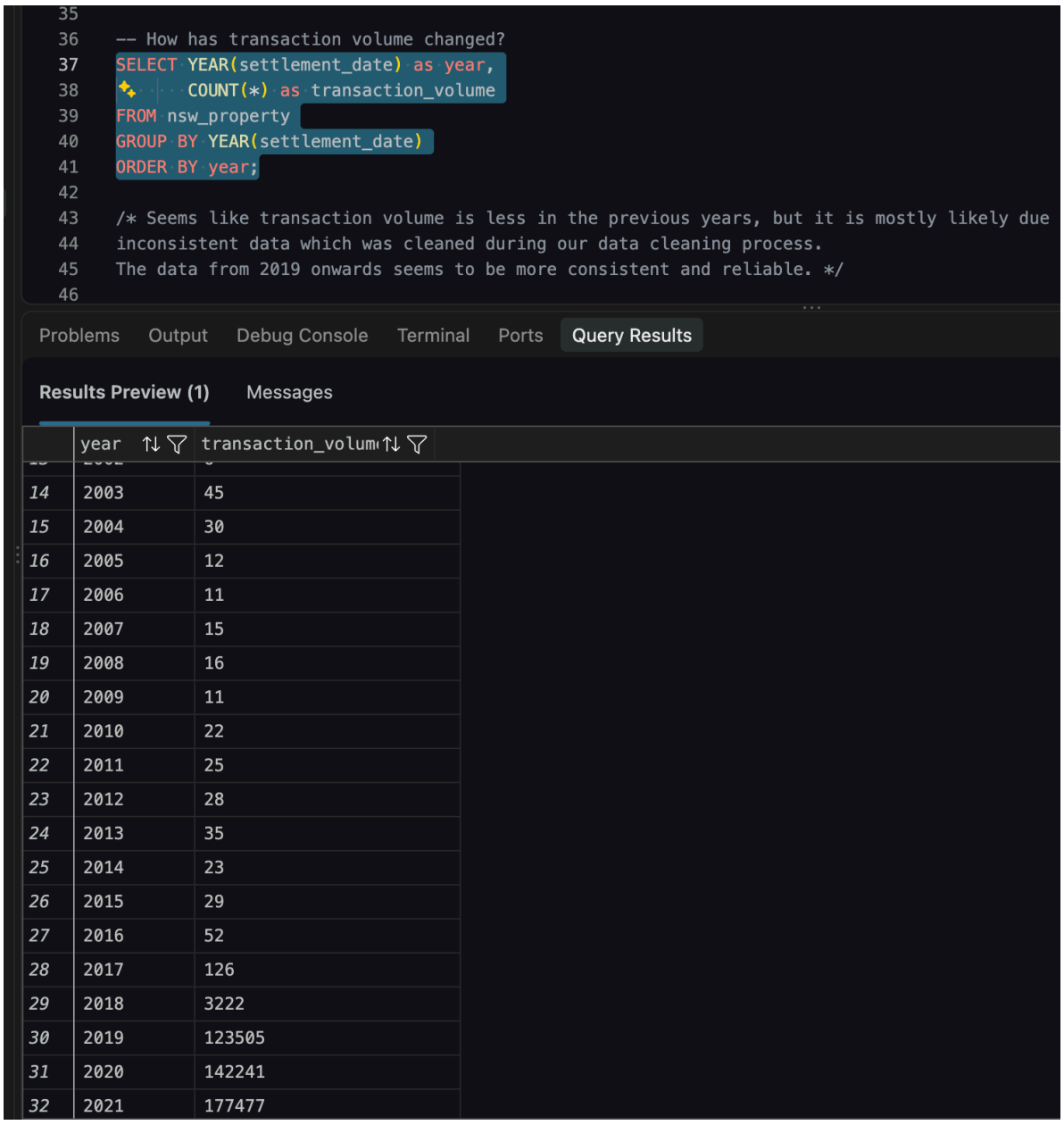
Changes in Transaction Volume

Transaction activity was also analysed by calculating the number of transactions recorded in each year.

```
SELECT
    YEAR(settlement_date) AS year,
    COUNT(*) AS transaction_volume
FROM nsw_property
GROUP BY YEAR(settlement_date)
ORDER BY year;
```

The results indicate that transaction volumes in the earlier years are lower than in the more recent period. However, this should be interpreted cautiously, as the lower volume may reflect inconsistencies or missing records in the earlier data rather than a genuine decline in NSW property activity.

Following the data-cleaning process, **data from 2019 onwards appears to be more consistent and reliable**, making the later period more suitable for identifying market trends.



```
35
36  -- How has transaction volume changed?
37  SELECT YEAR(settlement_date) as year,
38         COUNT(*) as transaction_volume
39  FROM nsw_property
40  GROUP BY YEAR(settlement_date)
41  ORDER BY year;
42
43  /* Seems like transaction volume is less in the previous years, but it is mostly likely due
44  inconsistent data which was cleaned during our data cleaning process.
45  The data from 2019 onwards seems to be more consistent and reliable. */
46
```

Problems | Output | Debug Console | Terminal | Ports | Query Results

Results Preview (1) | Messages

	year	transaction_volum
14	2003	45
15	2004	30
16	2005	12
17	2006	11
18	2007	15
19	2008	16
20	2009	11
21	2010	22
22	2011	25
23	2012	28
24	2013	35
25	2014	23
26	2015	29
27	2016	52
28	2017	126
29	2018	3222
30	2019	123505
31	2020	142241
32	2021	177477

2. Geographic Market Analysis

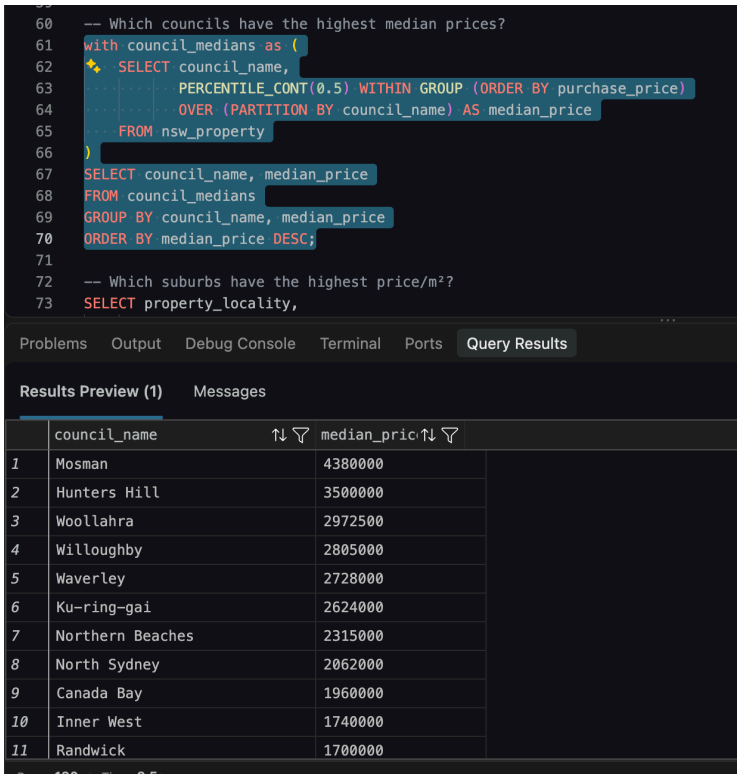
The next stage of the analysis examined the geographical distribution of property prices and transaction activity. This was particularly important because property values can vary significantly between councils, suburbs and postcodes.

Median Property Price by Council

Median purchase prices were calculated for each council to identify areas with the highest property values.

```
WITH council_medians AS (
    SELECT
        council_name,
        PERCENTILE_CONT(0.5)
            WITHIN GROUP (ORDER BY purchase_price)
            OVER (PARTITION BY council_name) AS
median_price
    FROM nsw_property
)
SELECT
    council_name,
    median_price
FROM council_medians
GROUP BY council_name, median_price
ORDER BY median_price DESC;
```

This analysis provides a high-level comparison of property values across NSW councils and helps identify the most expensive areas within the dataset.

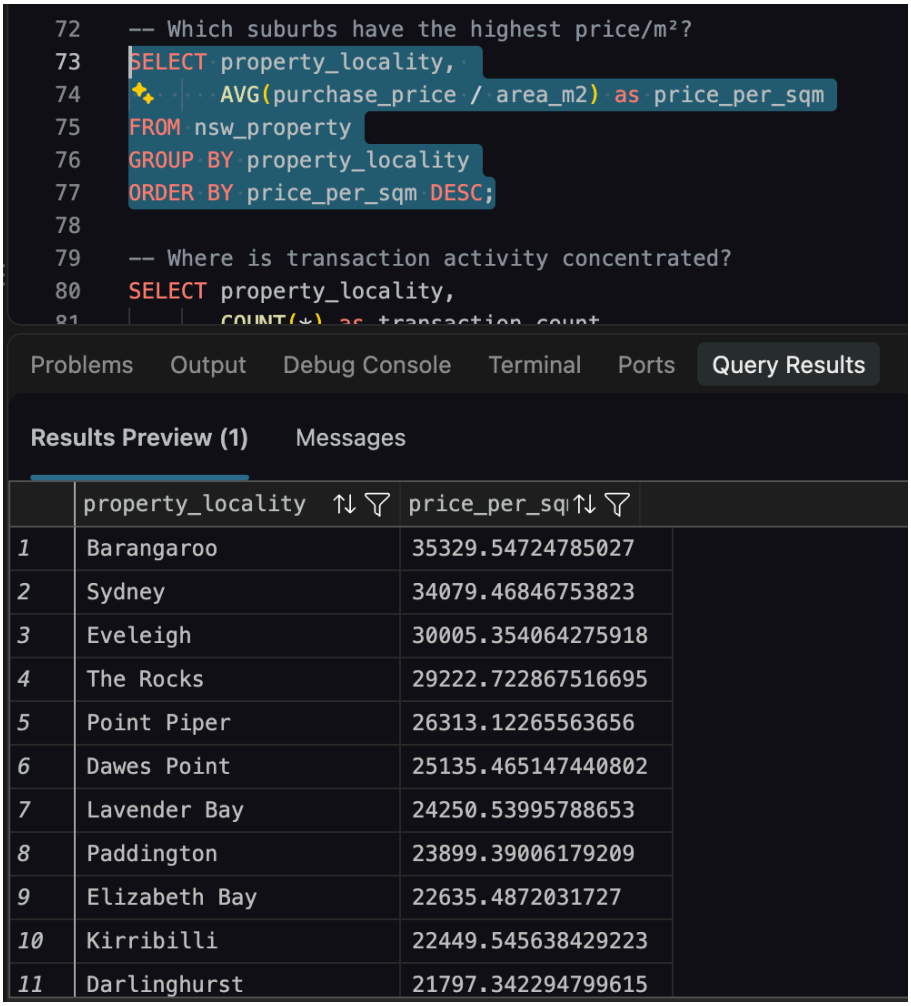


Price per Square Metre by Suburb

Average price per square metre was calculated to provide a more standardised measure of property value.

```
SELECT
    property_locality,
    AVG(purchase_price / area_m2) AS price_per_sqm
FROM nsw_property
GROUP BY property_locality
ORDER BY price_per_sqm DESC;
```

Price per square metre allows properties of different sizes to be compared more effectively. It provides an indication of how much buyers are paying for land or property space within different suburbs.



```
72  -- Which suburbs have the highest price/m²?
73  SELECT property_locality,
74      AVG(purchase_price / area_m2) as price_per_sqm
75  FROM nsw_property
76  GROUP BY property_locality
77  ORDER BY price_per_sqm DESC;
78
79  -- Where is transaction activity concentrated?
80  SELECT property_locality,
```

Problems | Output | Debug Console | Terminal | Ports | Query Results

Results Preview (1) | Messages

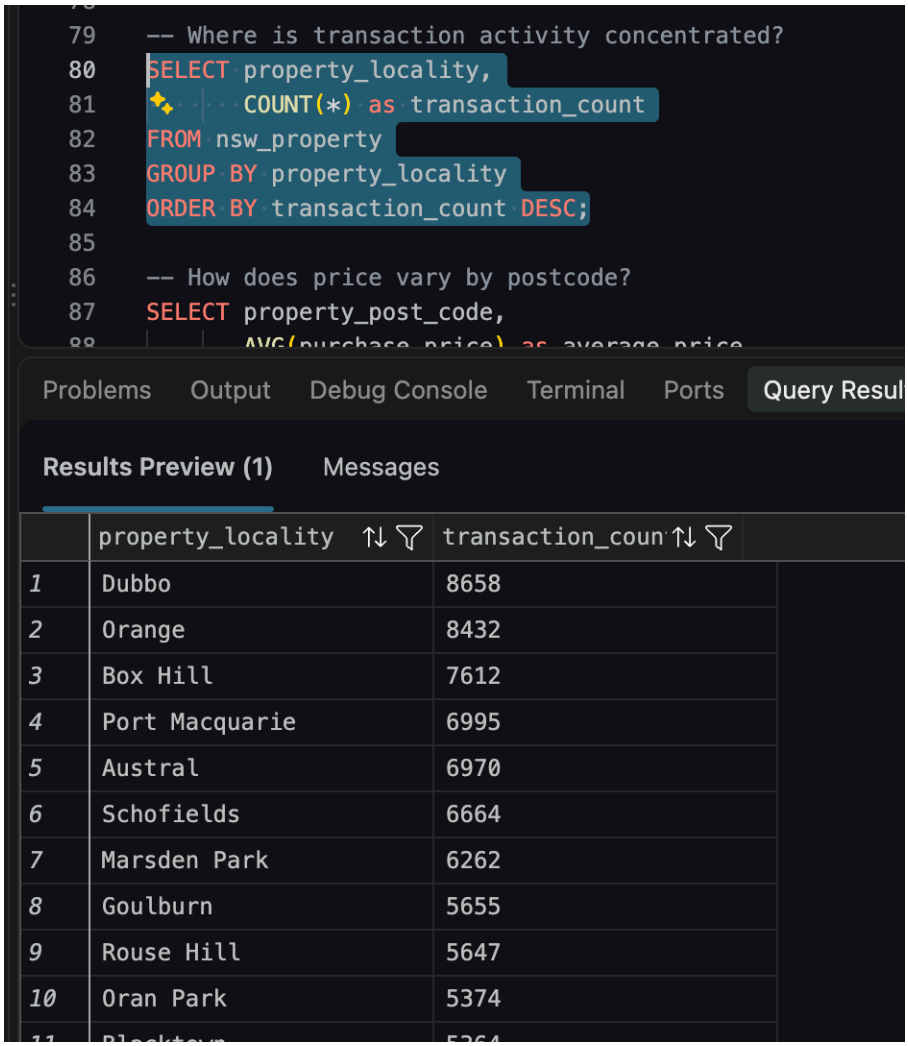
	property_locality	price_per_sq
1	Barangaroo	35329.54724785027
2	Sydney	34079.46846753823
3	Eveleigh	30005.354064275918
4	The Rocks	29222.722867516695
5	Point Piper	26313.12265563656
6	Dawes Point	25135.465147440802
7	Lavender Bay	24250.53995788653
8	Paddington	23899.39006179209
9	Elizabeth Bay	22635.4872031727
10	Kirribilli	22449.545638429223
11	Darlinghurst	21797.342294799615

Transaction Concentration

The number of transactions in each suburb was calculated to determine where property activity was most concentrated.

```
SELECT
    property_locality,
    COUNT(*) AS transaction_count
FROM nsw_property
GROUP BY property_locality
ORDER BY transaction_count DESC;
```

The results show where the greatest volume of property transactions occurred. Areas with high transaction counts can be considered particularly active property markets.



```
79  -- Where is transaction activity concentrated?
80  SELECT property_locality,
81         COUNT(*) as transaction_count
82  FROM nsw_property
83  GROUP BY property_locality
84  ORDER BY transaction_count DESC;
85
86  -- How does price vary by postcode?
87  SELECT property_post_code,
```

Problems | Output | Debug Console | Terminal | Ports | Query Resul

Results Preview (1) | Messages

	property_locality	transaction_coun
1	Dubbo	8658
2	Orange	8432
3	Box Hill	7612
4	Port Macquarie	6995
5	Austral	6970
6	Schofields	6664
7	Marsden Park	6262
8	Goulburn	5655
9	Rouse Hill	5647
10	Oran Park	5374

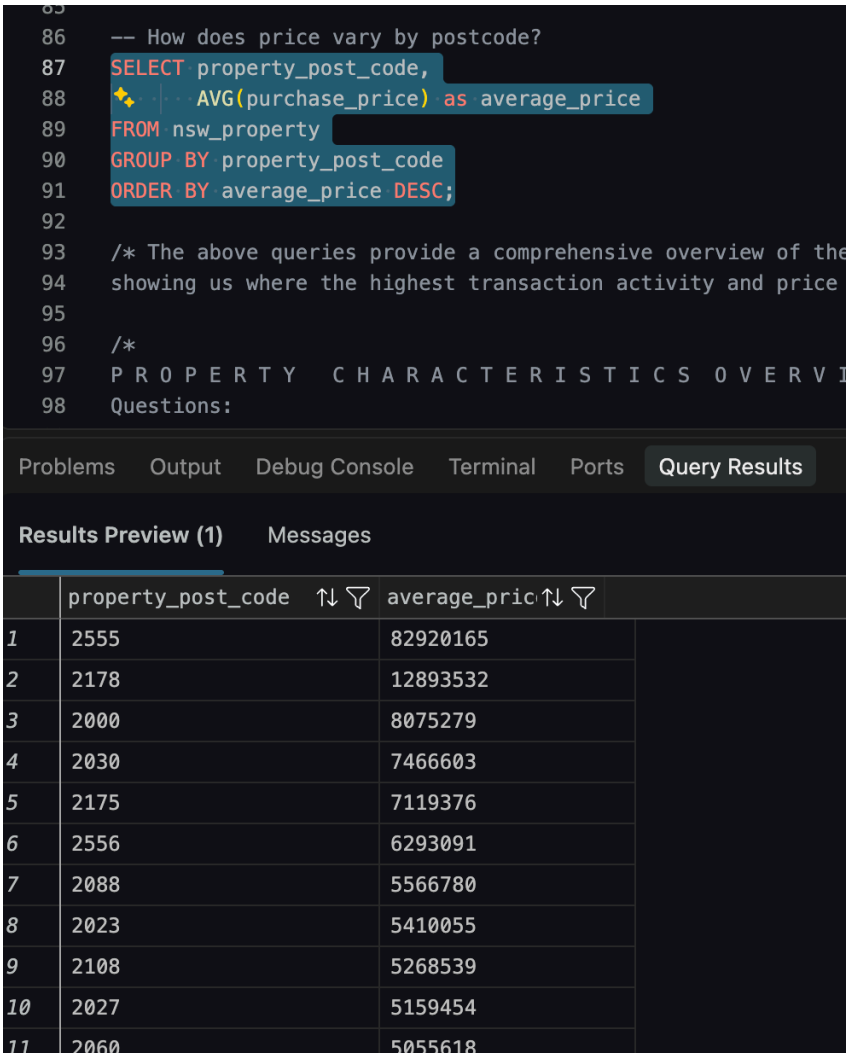
Average Price by Postcode

Average purchase prices were also compared across postcodes.

```
SELECT
    property_post_code,
    AVG(purchase_price) AS average_price
FROM nsw_property
GROUP BY property_post_code
ORDER BY average_price DESC;
```

This provides another geographical perspective on property prices and can highlight differences between local markets that may not be immediately visible when looking only at council-level results.

Overall, these queries provide a comprehensive geographical overview of the NSW property market, identifying **where transactions are concentrated and where the highest property values occur**.



```
-- How does price vary by postcode?
SELECT property_post_code,
       AVG(purchase_price) as average_price
FROM nsw_property
GROUP BY property_post_code
ORDER BY average_price DESC;

/* The above queries provide a comprehensive overview of the
showing us where the highest transaction activity and price

/*
PROPERTY CHARACTERISTICS OVERVI
Questions:
```

Problems | Output | Debug Console | Terminal | Ports | Query Results

Results Preview (1) | Messages

	property_post_code	average_pric
1	2555	82920165
2	2178	12893532
3	2000	8075279
4	2030	7466603
5	2175	7119376
6	2556	6293091
7	2088	5566780
8	2023	5410055
9	2108	5268539
10	2027	5159454
11	2060	5055618

3. Property Characteristics Analysis

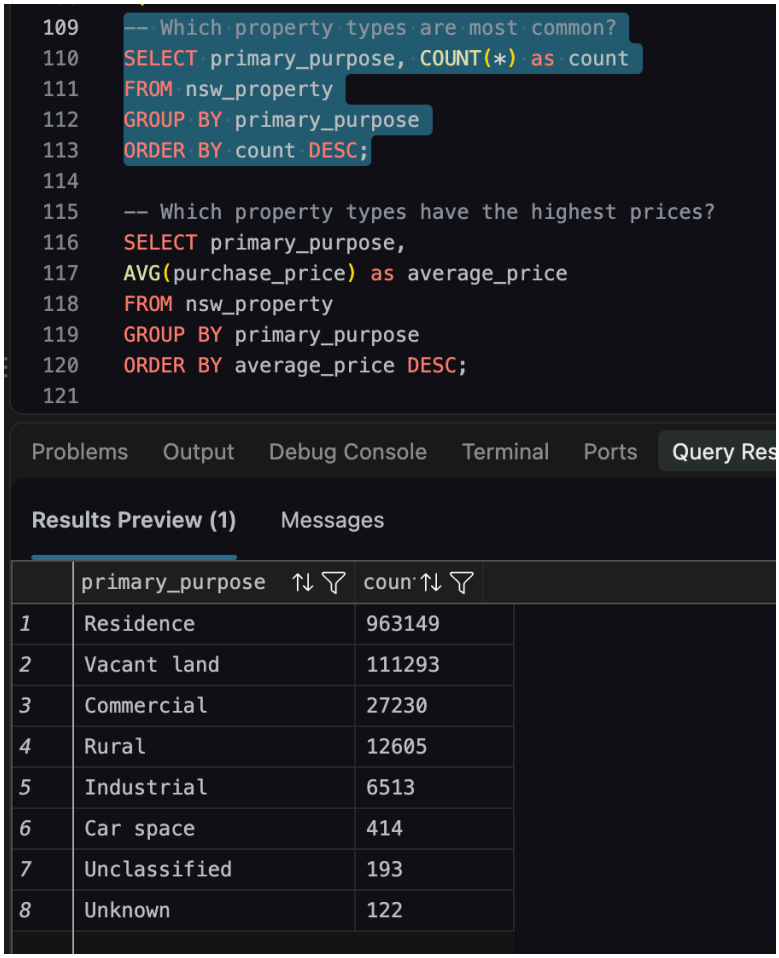
The third area of investigation focused on the characteristics of the properties themselves. The analysis examined property types, their associated prices, property size and price per square metre.

Most Common Property Types

The number of transactions for each property type was calculated using the `primary_purpose` field.

```
SELECT
    primary_purpose,
    COUNT(*) AS count
FROM nsw_property
GROUP BY primary_purpose
ORDER BY count DESC;
```

This identifies which property categories make up the largest proportion of the dataset and provides insight into the composition of the NSW property market.



```
-- Which property types are most common?
SELECT primary_purpose, COUNT(*) as count
FROM nsw_property
GROUP BY primary_purpose
ORDER BY count DESC;

-- Which property types have the highest prices?
SELECT primary_purpose,
AVG(purchase_price) as average_price
FROM nsw_property
GROUP BY primary_purpose
ORDER BY average_price DESC;
```

Problems | Output | Debug Console | Terminal | Ports | Query Res

Results Preview (1) | Messages

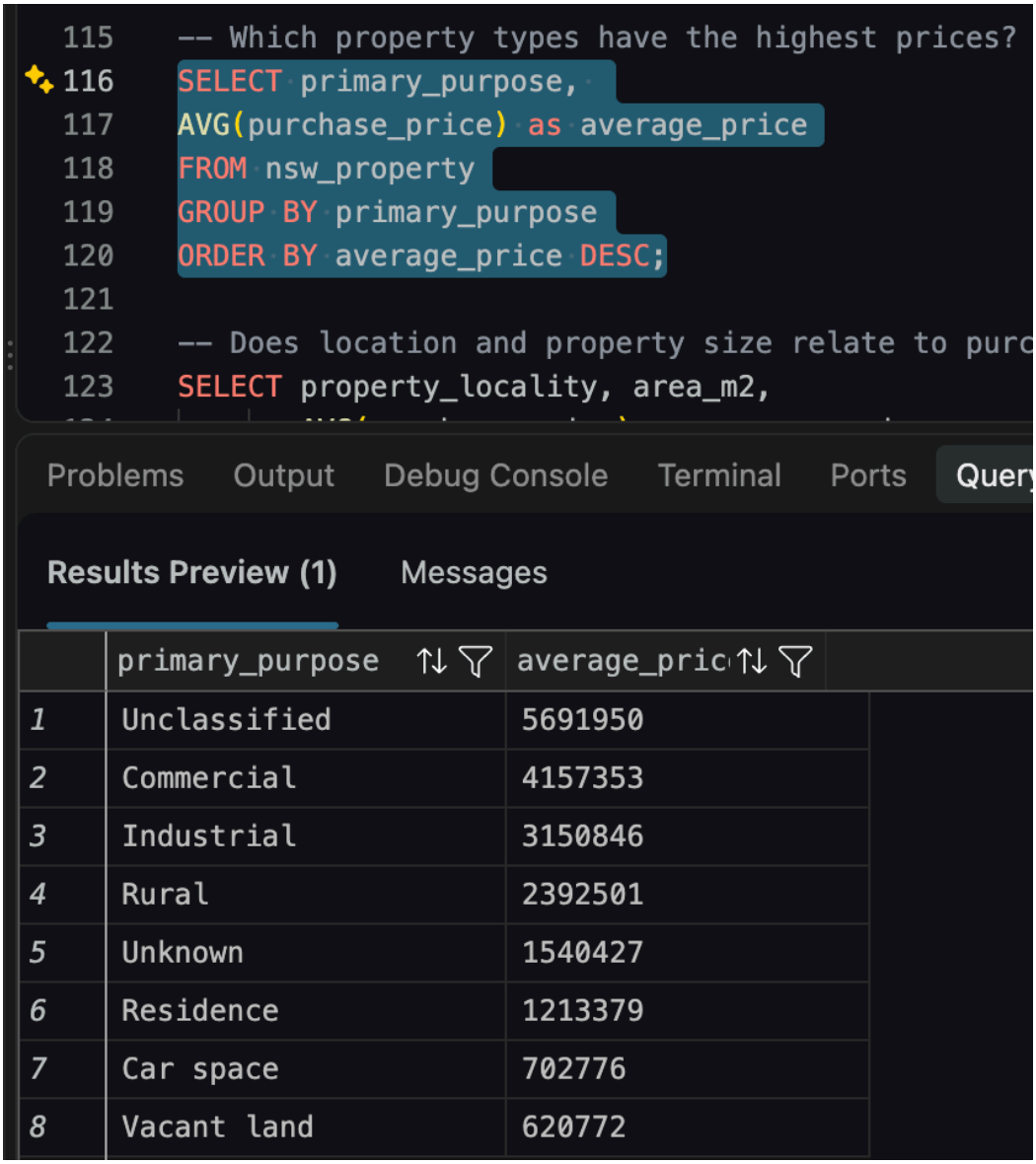
	primary_purpose	coun
1	Residence	963149
2	Vacant land	111293
3	Commercial	27230
4	Rural	12605
5	Industrial	6513
6	Car space	414
7	Unclassified	193
8	Unknown	122

Average Price by Property Type

Average purchase prices were then compared between property types.

```
SELECT
    primary_purpose,
    AVG(purchase_price) AS average_price
FROM nsw_property
GROUP BY primary_purpose
ORDER BY average_price DESC;
```

This analysis highlights differences in the typical purchase price between residential, commercial, industrial and other property categories.



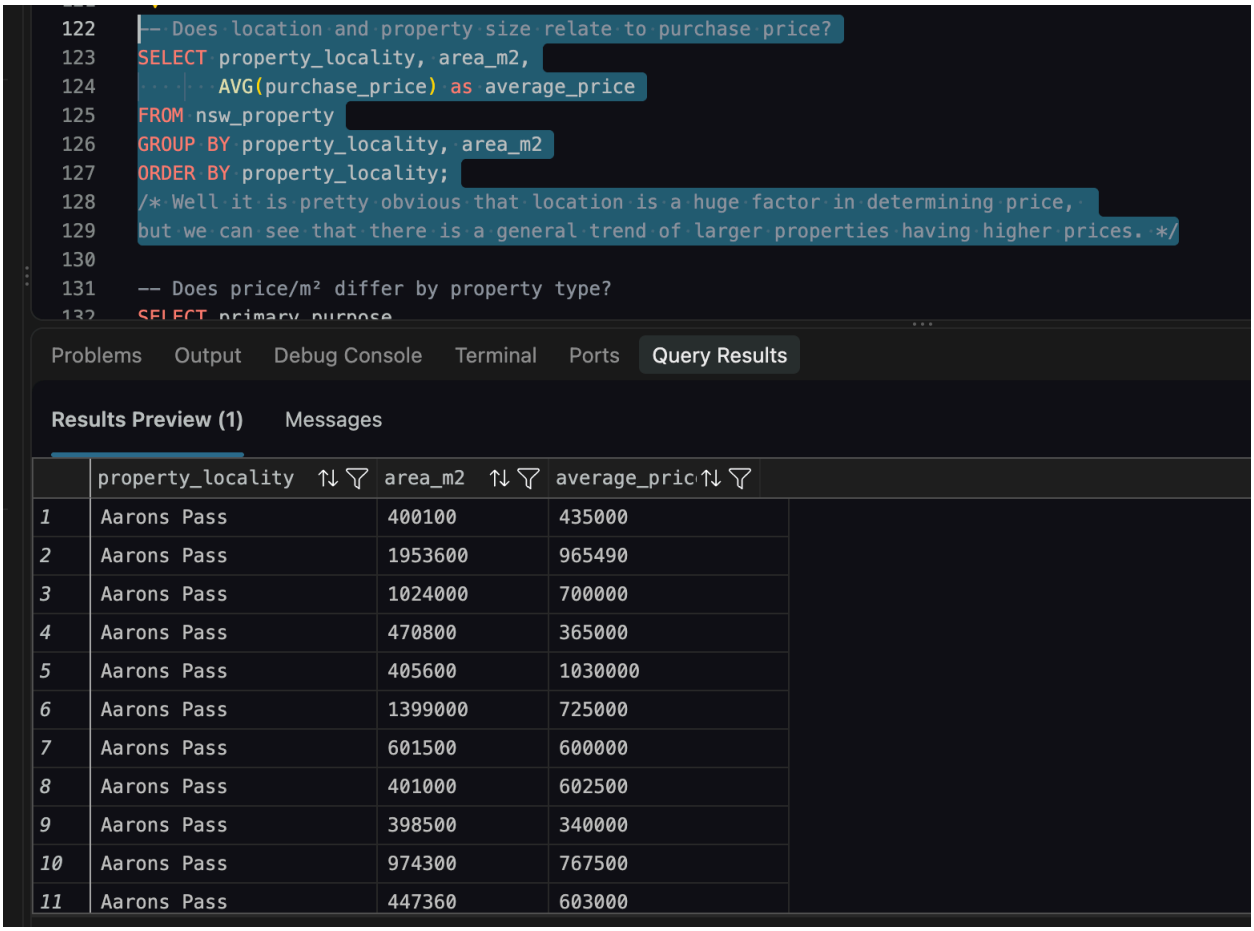
Relationship Between Property Size, Location and Price

To investigate whether property size and location are associated with higher purchase prices, average prices were grouped by both locality and property area.

```
SELECT
    property_locality,
    area_m2,
    AVG(purchase_price) AS average_price
FROM nsw_property
GROUP BY property_locality, area_m2
ORDER BY property_locality;
```

The results suggest that **location is a major factor influencing property prices**. There is also a general relationship between property size and purchase price, with larger properties tending to have higher total purchase prices.

However, this relationship should not be interpreted as a simple cause-and-effect relationship, as factors such as location, property type and local demand can also have a significant influence on price.



	property_locality	area_m2	average_pric
1	Aarons Pass	400100	435000
2	Aarons Pass	1953600	965490
3	Aarons Pass	1024000	700000
4	Aarons Pass	470800	365000
5	Aarons Pass	405600	1030000
6	Aarons Pass	1399000	725000
7	Aarons Pass	601500	600000
8	Aarons Pass	401000	602500
9	Aarons Pass	398500	340000
10	Aarons Pass	974300	767500
11	Aarons Pass	447360	603000

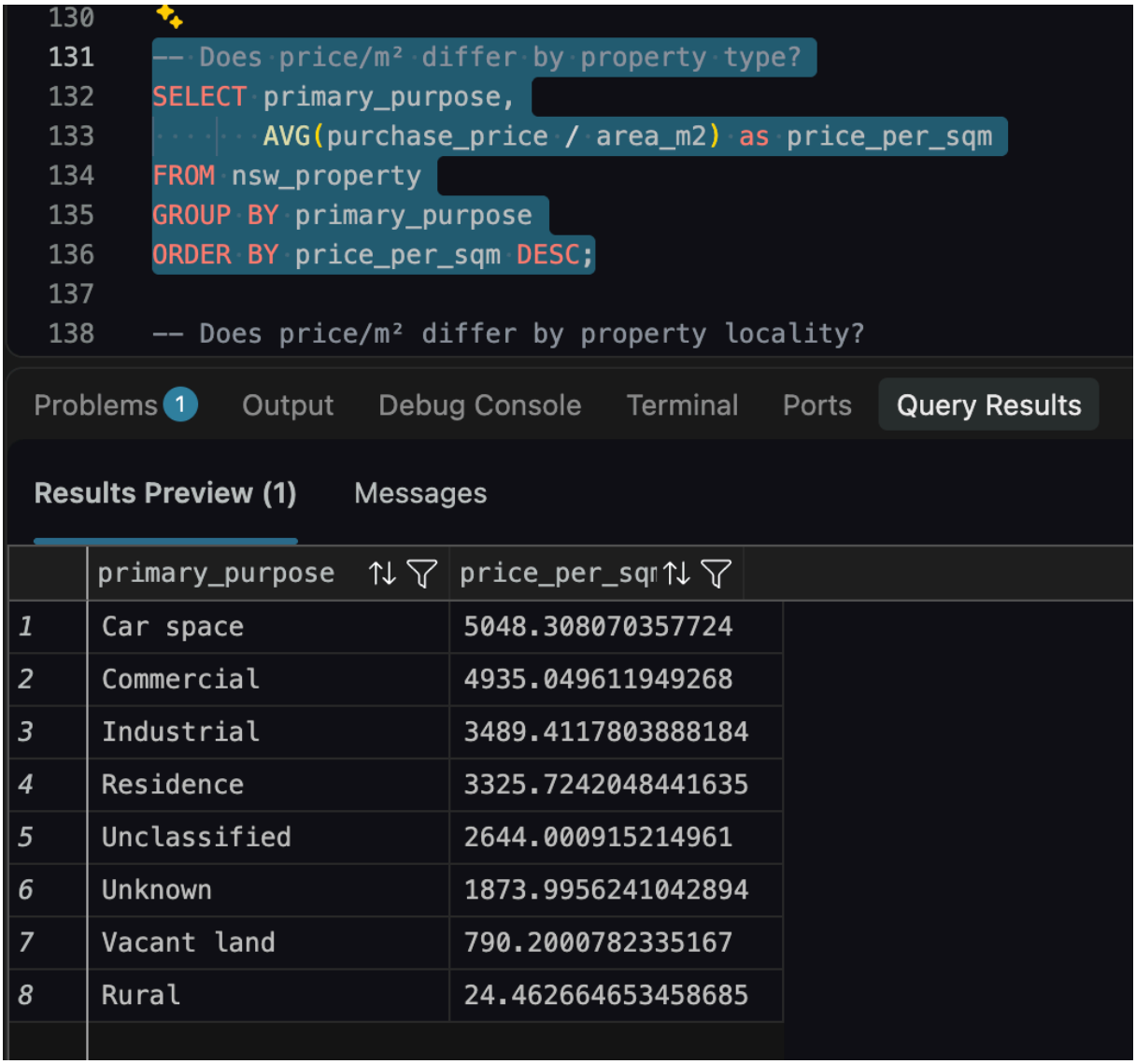
Price per Square Metre by Property Type

Price per square metre was calculated for each property type to allow a more meaningful comparison between different categories.

```
SELECT
    primary_purpose,
    AVG(purchase_price / area_m2) AS price_per_sqm
FROM nsw_property
GROUP BY primary_purpose
ORDER BY price_per_sqm DESC;
```

An interesting finding was that **car spaces recorded a particularly high average price per square metre**, exceeding commercial and industrial properties.

This result is likely influenced by the nature of car spaces. Individual car spaces typically have a very small recorded area, meaning that dividing the purchase price by area can produce a very high price-per-square-metre figure. Car spaces can also command high prices in locations where parking is scarce and demand is strong.



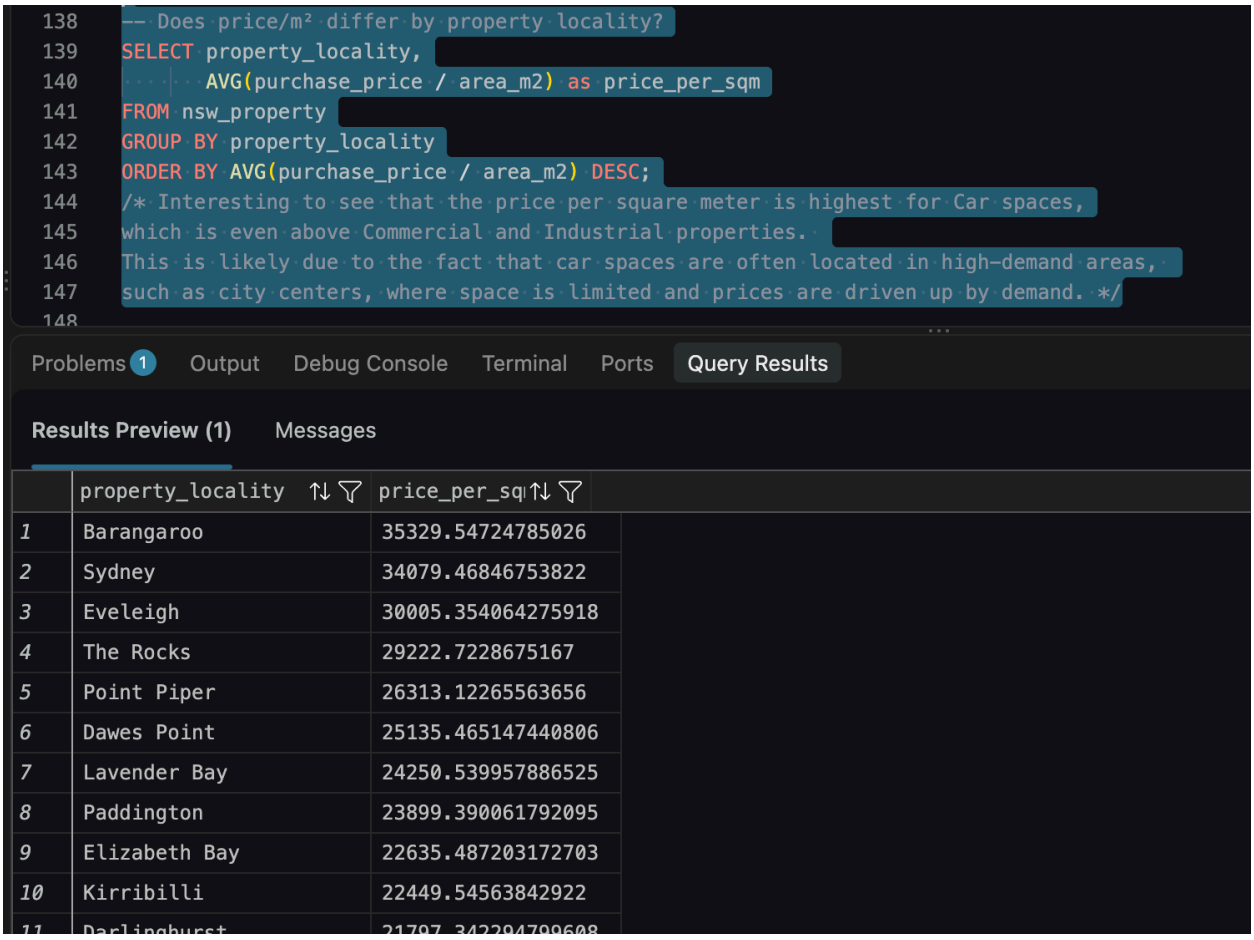
Therefore, while price per square metre is useful for comparison, the results need to be interpreted in the context of the property type.

Price per Square Metre by Locality

The analysis was also extended to individual suburbs.

```
SELECT
    property_locality,
    AVG(purchase_price / area_m2) AS price_per_sqm
FROM nsw_property
GROUP BY property_locality
ORDER BY AVG(purchase_price / area_m2) DESC;
```

This provides further insight into areas where property space commands the highest prices.



```
-- Does price/m² differ by property locality?
SELECT property_locality,
       AVG(purchase_price / area_m2) as price_per_sqm
FROM nsw_property
GROUP BY property_locality
ORDER BY AVG(purchase_price / area_m2) DESC;
/* Interesting to see that the price per square meter is highest for Car spaces,
which is even above Commercial and Industrial properties.
This is likely due to the fact that car spaces are often located in high-demand areas,
such as city centers, where space is limited and prices are driven up by demand. */
```

Problems 1 | Output | Debug Console | Terminal | Ports | Query Results

Results Preview (1) | Messages

	property_locality	price_per_sq
1	Barangaroo	35329.54724785026
2	Sydney	34079.46846753822
3	Eveleigh	30005.354064275918
4	The Rocks	29222.7228675167
5	Point Piper	26313.12265563656
6	Dawes Point	25135.465147440806
7	Lavender Bay	24250.539957886525
8	Paddington	23899.390061792095
9	Elizabeth Bay	22635.487203172703
10	Kirribilli	22449.54563842922
11	Darlinghurst	21797.342294799608

4. Market Dynamics

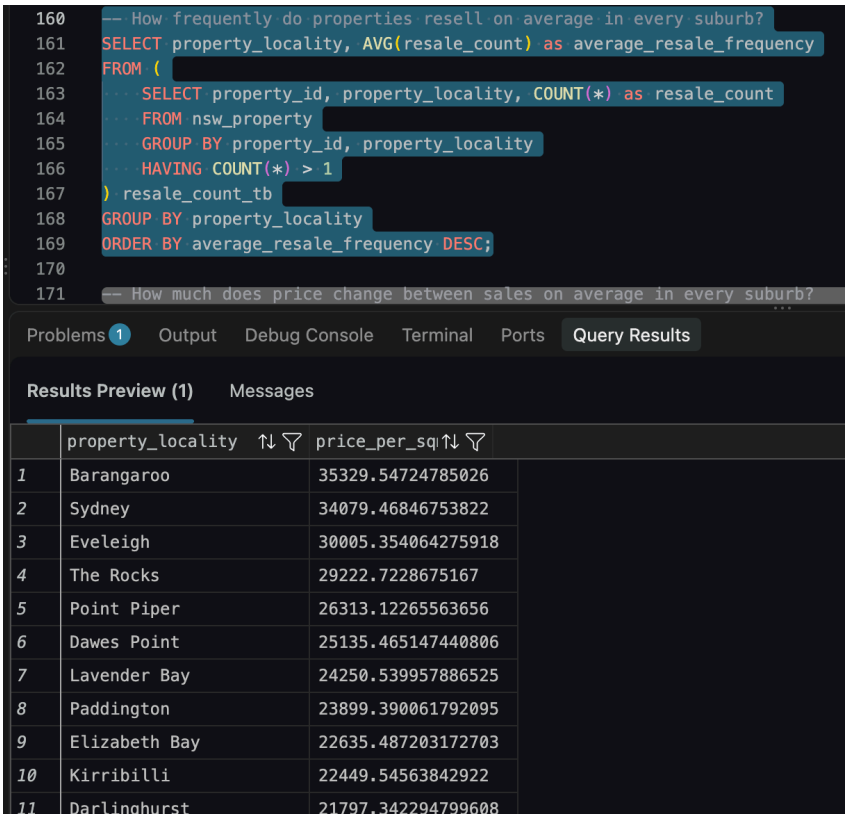
The final stage of the analysis investigated the dynamics of the property market, rather than simply looking at individual transaction prices. Specifically, the analysis examined property resales, changes in prices between transactions, and the time between contract and settlement.

Property Resale Frequency

To investigate how frequently properties were resold, properties with more than one transaction were identified and the average number of transactions per property was calculated for each suburb.

```
SELECT
    property_locality,
    AVG(resale_count) AS average_resale_frequency
FROM (
    SELECT
        property_id,
        property_locality,
        COUNT(*) AS resale_count
    FROM nsw_property
    GROUP BY property_id, property_locality
    HAVING COUNT(*) > 1
) resale_count_tb
GROUP BY property_locality
ORDER BY average_resale_frequency DESC;
```

This provides an indication of which areas experience more frequent repeat transactions and therefore potentially greater property turnover.

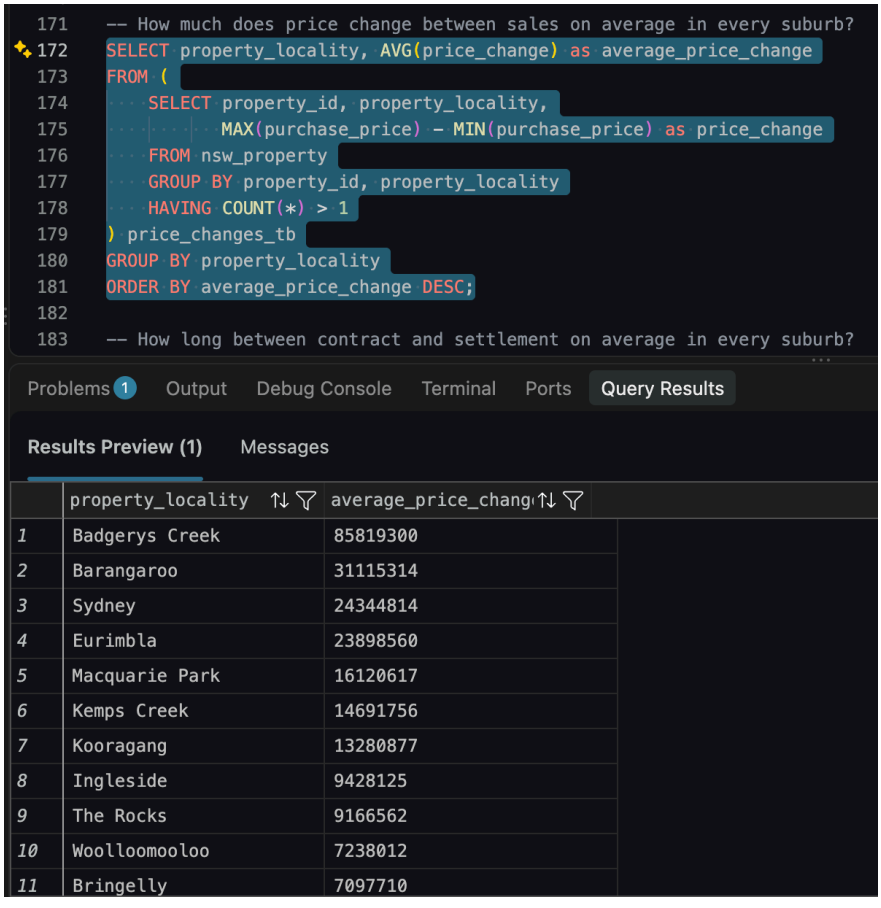


Change in Price Between Sales

The next analysis examined how much property prices changed between transactions.

```
SELECT
    property_locality,
    AVG(price_change) AS average_price_change
FROM (
    SELECT
        property_id,
        property_locality,
        MAX(purchase_price) - MIN(purchase_price) AS
price_change
    FROM nsw_property
    GROUP BY property_id, property_locality
    HAVING COUNT(*) > 1
) price_changes_tb
GROUP BY property_locality
ORDER BY average_price_change DESC;
```

The analysis identified **Badgerys Creek** as an area with particularly large price changes between recorded sales. This may reflect significant development and infrastructure activity in the area, including the development surrounding Western Sydney International Airport.



```
171  -- How much does price change between sales on average in every suburb?
172  SELECT property_locality, AVG(price_change) as average_price_change
173  FROM (
174      SELECT property_id, property_locality,
175          MAX(purchase_price) - MIN(purchase_price) as price_change
176      FROM nsw_property
177      GROUP BY property_id, property_locality
178      HAVING COUNT(*) > 1
179  ) price_changes_tb
180  GROUP BY property_locality
181  ORDER BY average_price_change DESC;
182
183  -- How long between contract and settlement on average in every suburb?
```

Problems 1 | Output | Debug Console | Terminal | Ports | Query Results

Results Preview (1) | Messages

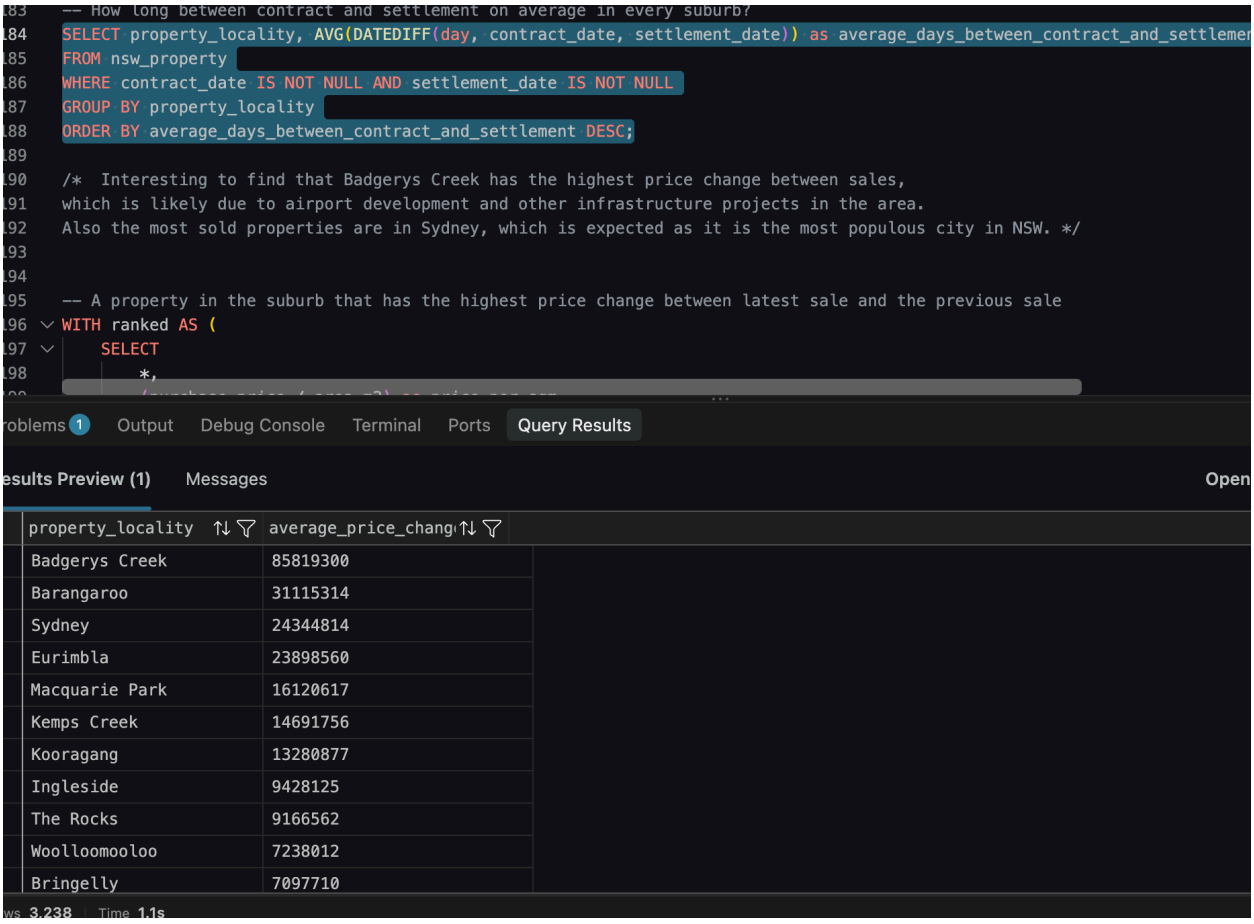
	property_locality	average_price_chang
1	Badgerys Creek	85819300
2	Barangaroo	31115314
3	Sydney	24344814
4	Eurimbla	23898560
5	Macquarie Park	16120617
6	Kemps Creek	14691756
7	Kooragang	13280877
8	Ingleside	9428125
9	The Rocks	9166562
10	Woolloomooloo	7238012
11	Bringelly	7097710

Contract-to-Settlement Period

The average number of days between contract and settlement was calculated for each suburb.

```
SELECT
    property_locality,
    AVG(DATEDIFF(day, contract_date, settlement_date))
        AS average_days_between_contract_and_settlement
FROM nsw_property
WHERE contract_date IS NOT NULL
  AND settlement_date IS NOT NULL
GROUP BY property_locality
ORDER BY average_days_between_contract_and_settlement DESC;
```

This analysis provides insight into the typical time required for transactions to progress from contract signing to settlement and whether settlement periods vary across different locations.



5. Property-Level Resale Analysis

A more detailed analysis was then performed to identify individual properties experiencing the largest increases in value between recorded transactions.

The query uses several Common Table Expressions (CTEs) to create a property-level price history.

First, duplicate transaction records are identified and removed:

```
ROW_NUMBER() OVER (
    PARTITION BY property_id,
        property_unit_number,
        property_house_number,
        property_street_name,
        property_locality,
        settlement_date,
        area_m2
    ORDER BY purchase_price DESC
) AS rn
```

The cleaned records are then used to construct a transaction history for each property. The `LAG()` function allows the previous sale price and settlement date to be compared against the latest transaction.

```
LAG(settlement_date) OVER (
    PARTITION BY property_id
    ORDER BY settlement_date, contract_date
) AS previous_settlement_date,

LAG(purchase_price) OVER (
    PARTITION BY property_id
    ORDER BY settlement_date, contract_date
) AS previous_price
```

Finally, the absolute and percentage change between transactions are calculated:

```
latest_price - previous_price AS price_change,

(((latest_price - previous_price) * 1.0)
 / (previous_price * 1.0)) * 100.0
 AS price_change_percentage
```

```sql
SELECT
    primary_purpose,
    contract_date,
    settlement_date,
    price_per_sqm,
    previous_price_per_sqm,
    previous_settlement_date,
    previous_price,
    latest_price,
    latest_price - previous_price AS price_change,
    (((latest_price - previous_price)*1.0) / (previous_price*1.0)) * 100.0 AS price_change_percentage
FROM price_history
WHERE previous_price IS NOT NULL
--AND property_locality = 'Badgerys Creek'
ORDER BY price_change DESC;

/* This query removes duplicate transaction records by selecting the highest purchase price for each property
and settlement date. It then calculates the price per square metre and compares the latest sale with the previous sale
for each property, including the change in sale price and percentage change. The results can be filtered
by the suburb of interest and are ordered by price change in descending order, placing the properties
with the largest price increases at the top of the result set.*/
```

Problems 1 | Output | Debug Console | Terminal | Ports | Query Results

Results Preview (1) | Messages | Open in New Tab

	property_id	property_unit_number	property_house_number	property_street_name	property_locality	property_post
1	2032103	NULL	339	Kent St	Sydney	2000
2	3791727	NULL	197	Herring Rd	Macquarie Park	2113
3	3207271	NULL	61	Phillip St	Sydney	2000
4	2545482	NULL	1669 A	Elizabeth Dr	Badgerys Creek	2555
5	3611416	NULL	3	Broadcast Way	Artarmon	2064
6	4074476	NULL	421	The Northern Rd	Cobbitty	2570
7	2033517	NULL	46	King St	Sydney	2000
8	2150986	NULL	174	William St	Woolloomooloo	2011
9	4449147	NULL	149	Orchard Rd	Chester Hill	2162
10	4355630	6501	1 A	Barangaroo Ave	Barangaroo	2000
11	4521844	NULL	11	Coolong Rd	Vaucluse	2030

This query provides a more detailed view of property appreciation by comparing a property's latest recorded sale with its previous sale. The results can also be filtered to a particular suburb, such as Badgerys Creek, allowing individual properties within a high-growth area to be investigated.

The use of `LAG()` is particularly useful here because it allows the analysis to move beyond simple aggregate statistics and examine **individual property price histories**.

The final results are ordered by `price_change` in descending order, placing the properties with the largest absolute increases at the top of the result set.

6. Key Findings from the SQL Analysis

The SQL analysis provided several important insights into the NSW property market:

- **Market activity:** Transaction volumes vary considerably across the dataset, with data from 2019 onwards appearing more consistent and reliable following the data-cleaning process.

- **Price trends:** Median purchase prices provide a more robust measure of market movement than average prices because of the large variation in property values.
- **Geographical differences:** Property prices and transaction activity vary substantially between councils, suburbs and postcodes.
- **Location matters:** The analysis indicates that location is a major factor associated with purchase price, with some suburbs commanding substantially higher prices than others.
- **Property size:** Larger properties generally have higher total purchase prices, although the relationship is influenced by location and property type.
- **Price per square metre:** Price per square metre varies significantly between property types and locations. Car spaces produced particularly high values, demonstrating the importance of considering property characteristics when interpreting this metric.
- **Resale activity:** Some suburbs show considerably higher levels of repeat property transactions than others.
- **Property appreciation:** Badgerys Creek showed particularly large price changes between recorded sales, potentially reflecting development and infrastructure investment in the area.
- **Transaction timing:** The time between contract and settlement also varies geographically, providing another dimension through which market behaviour can be assessed.
- **Individual property analysis:** Using `LAG()` made it possible to compare consecutive sales of the same property and identify properties with the largest increases in value.

5. Interactive Executive Dashboard in Tableau

First of all we need to do some processing of the data we got from the Property Level Resale Analysis, since it doesn't have Latitude and Longitude values for the individual properties we will use python to map them using data from https://www.data.gov.au/data/dataset/geocoded-national-address-file-g-naf/. Here is the short snippet of the code used for doing the same.

[5]:
```
import pandas as pd

GNAF = "./GNAF/Standard/"   # to find accurate geo path for mapping
#we use data from the website
#https://www.data.gov.au/data/dataset/geocoded-national-address-file-g-naf/

# --- Load only what's needed from G-NAF ---
addr = pd.read_csv(GNAF + "NSW_ADDRESS_DETAIL_psv.psv", sep="|", dtype=str,
                   usecols=["ADDRESS_DETAIL_PID", "LOCALITY_PID", "POSTCODE", "DATE_RETIRED"])
addr = addr[addr["DATE_RETIRED"].isna()].drop(columns="DATE_RETIRED")

geo = pd.read_csv(GNAF + "NSW_ADDRESS_DEFAULT_GEOCODE_psv.psv", sep="|", dtype=str,
                  usecols=["ADDRESS_DETAIL_PID", "LATITUDE", "LONGITUDE", "DATE_RETIRED"])
geo = geo[geo["DATE_RETIRED"].isna()].drop(columns="DATE_RETIRED")

loc = pd.read_csv(GNAF + "NSW_LOCALITY_psv.psv", sep="|", dtype=str,
                  usecols=["LOCALITY_PID", "LOCALITY_NAME"])

gnaf = addr.merge(geo, on="ADDRESS_DETAIL_PID").merge(loc, on="LOCALITY_PID")
gnaf["LATITUDE"] = gnaf["LATITUDE"].astype(float)
gnaf["LONGITUDE"] = gnaf["LONGITUDE"].astype(float)

def clean(s):
    return (s.fillna("").astype(str).str.upper().str.strip()
             .str.replace(r"\.0$", "", regex=True)
             .str.replace(r"[^A-Z0-9 ]", "", regex=True)
             .str.replace(r"\s+", " ", regex=True))

gnaf["k_suburb"] = clean(gnaf["LOCALITY_NAME"])
gnaf["k_post"] = clean(gnaf["POSTCODE"])

places = (gnaf.groupby(["k_suburb", "k_post"], as_index=False)[["LATITUDE", "LONGITUDE"]]
              .median()
              .rename(columns={"LATITUDE": "latitude", "LONGITUDE": "longitude"}))
```

```
# --- Your data ---
df = pd.read_csv("nsw_property_sales_sql_output.csv", dtype=str)
df = df.drop(columns=["latitude", "longitude"], errors="ignore")

df["k_suburb"] = clean(df["property_locality"])
df["k_post"] = clean(df["property_post_code"])

df = df.merge(places, on=["k_suburb", "k_post"], how="left")
print(f"Matched: {df['latitude'].notna().mean():.1%}")

df = df.drop(columns=["k_suburb", "k_post"])
df.duplicated().sum()
#df.to_csv("nsw_properties_geocoded.csv", index=False)
```

```
Matched: 100.0%
```

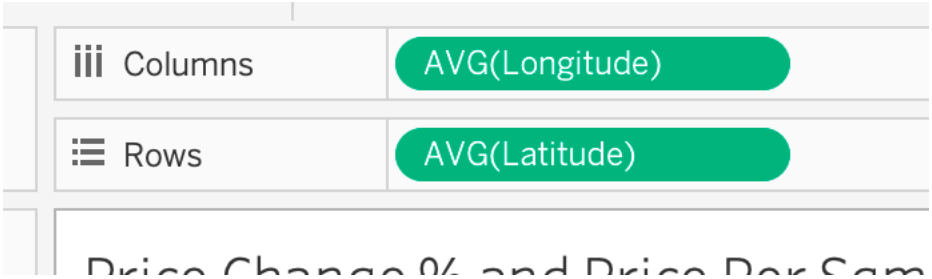
```
np.int64(0)
```

Now we will load this data into Tableau for our Interactive executive dashboard.

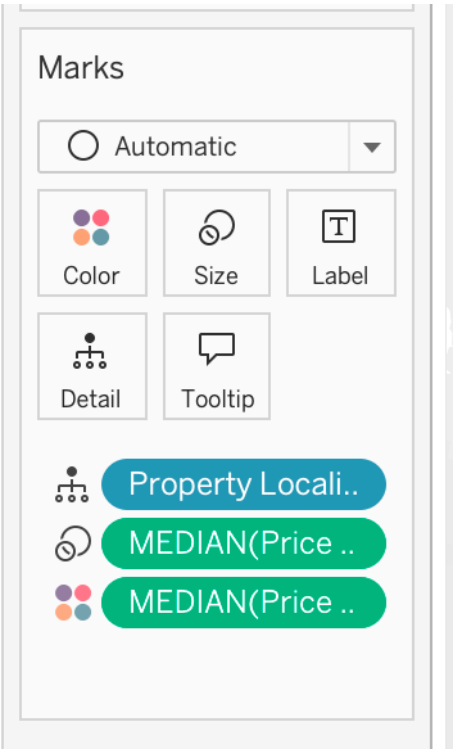
Components in Tableau:
Executive KPI Summary Cards (Price Change %, Price Per Sqm, Total Transactions)

Price Change %	Price Per Sqm	Total Transactions
12.26	4,400	136,937

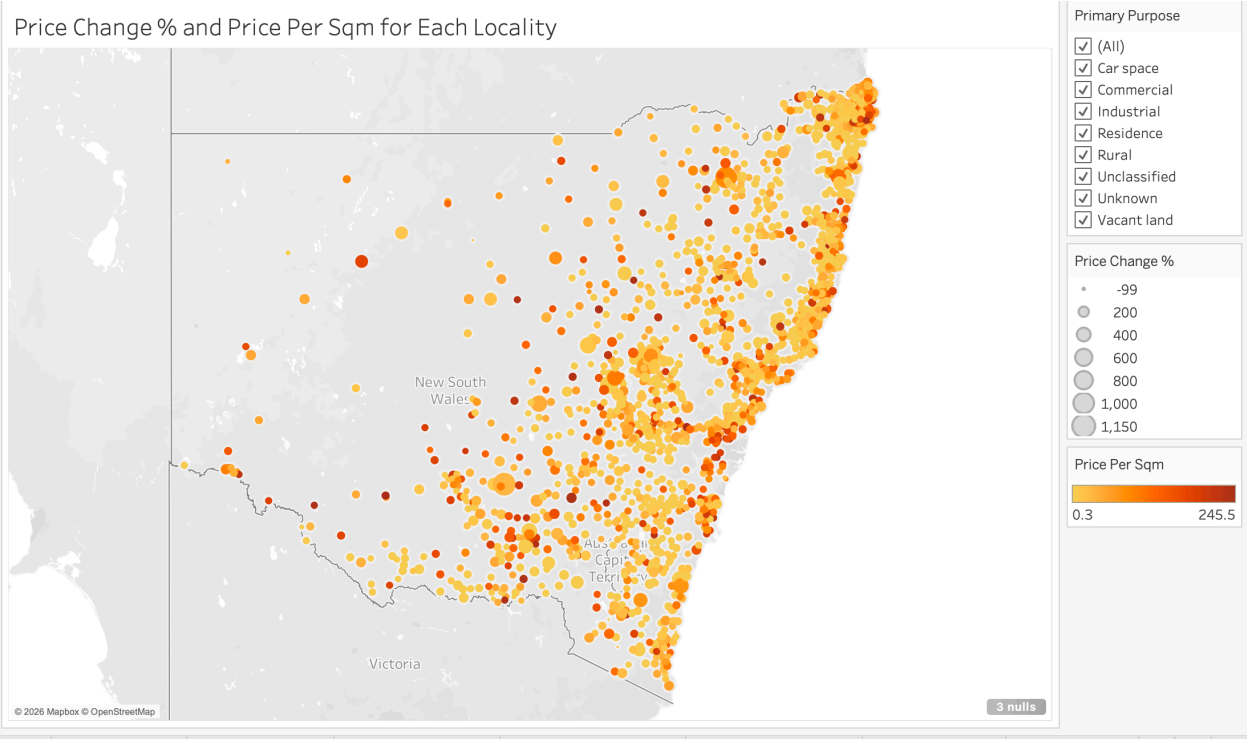
Interactive Spatial Heatmap (NSW map colored by Price Change % and $/m^2)



First we shall set the longitude and latitude to column and rows respectively and after that we will add property locality in the marks section as Detail so the points show up on the map. We will set Price Change % as size to control the size of the dot and Price/m^2 as colour so we can detect which areas have high Price/m^2 .

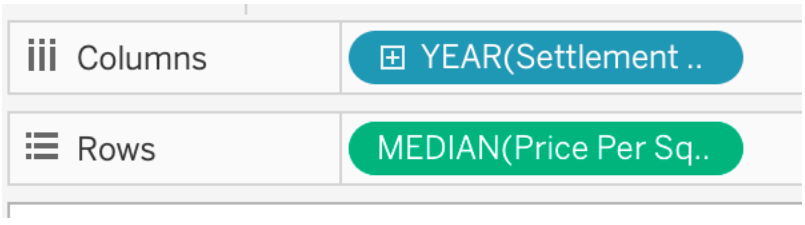


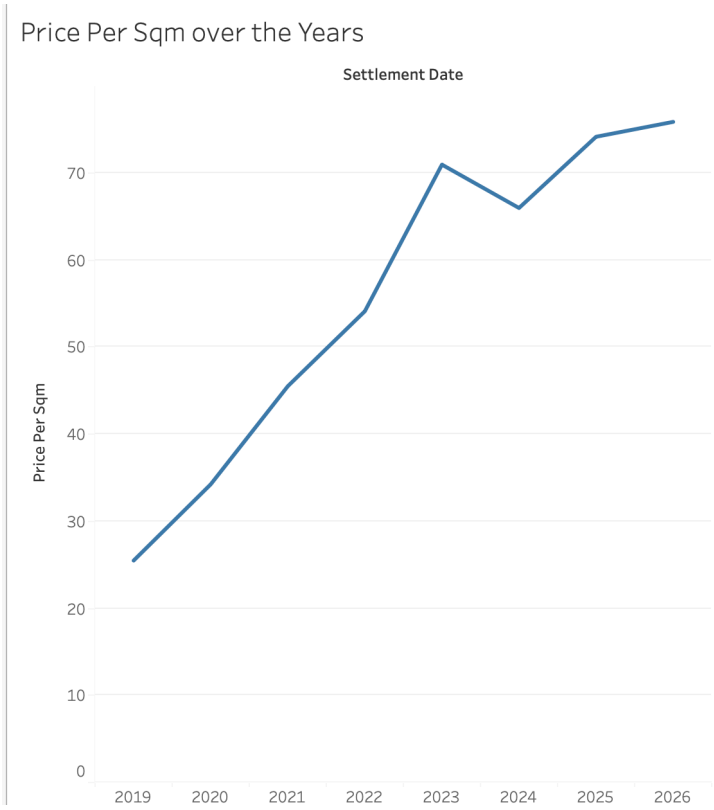
We will also add a filter based on Primary Purpose so users can select which type of property they need information for. Now this is how our map looks like after all the changes.



Multi-Year Trend Line (Price/m^2 movement from 2019 to 2026

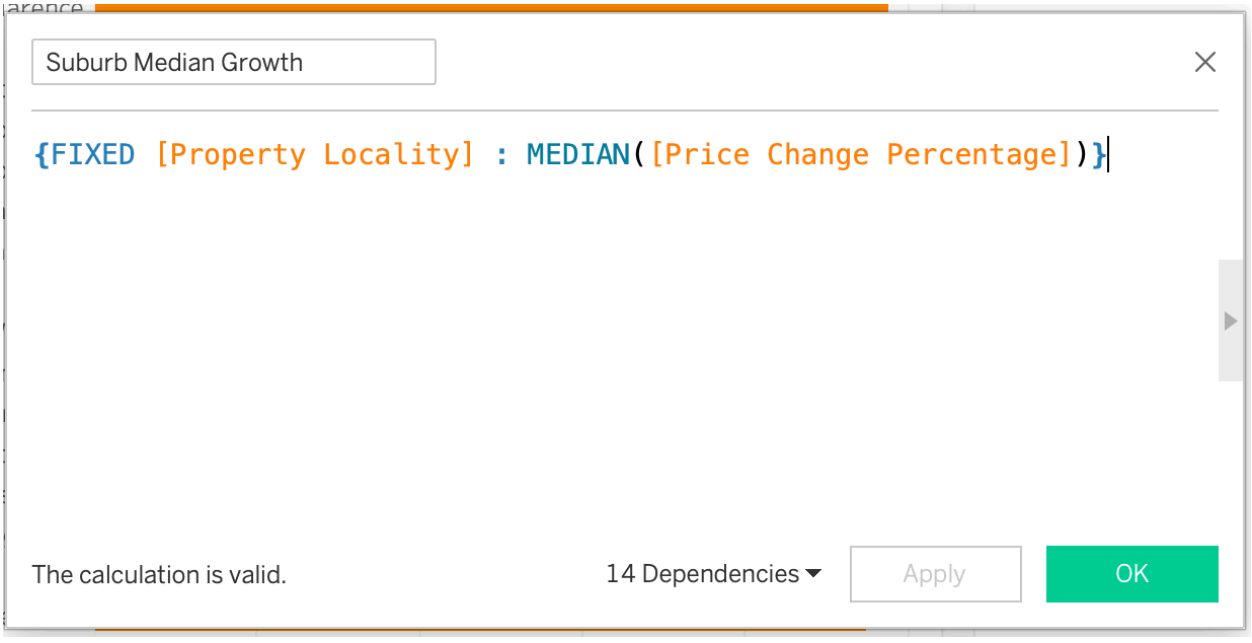
First of all we will set the column as Year from Settlement Year and then the Median of Price/m^2 as the row. This will give us a Line Chart. We will also add a filter to only get years from 2019 to 2026 since we have good amount of transactions present in those years as we checked in the data cleaning process.





Suburb Ranking Bar Chart (Ranked by Growth or Value)

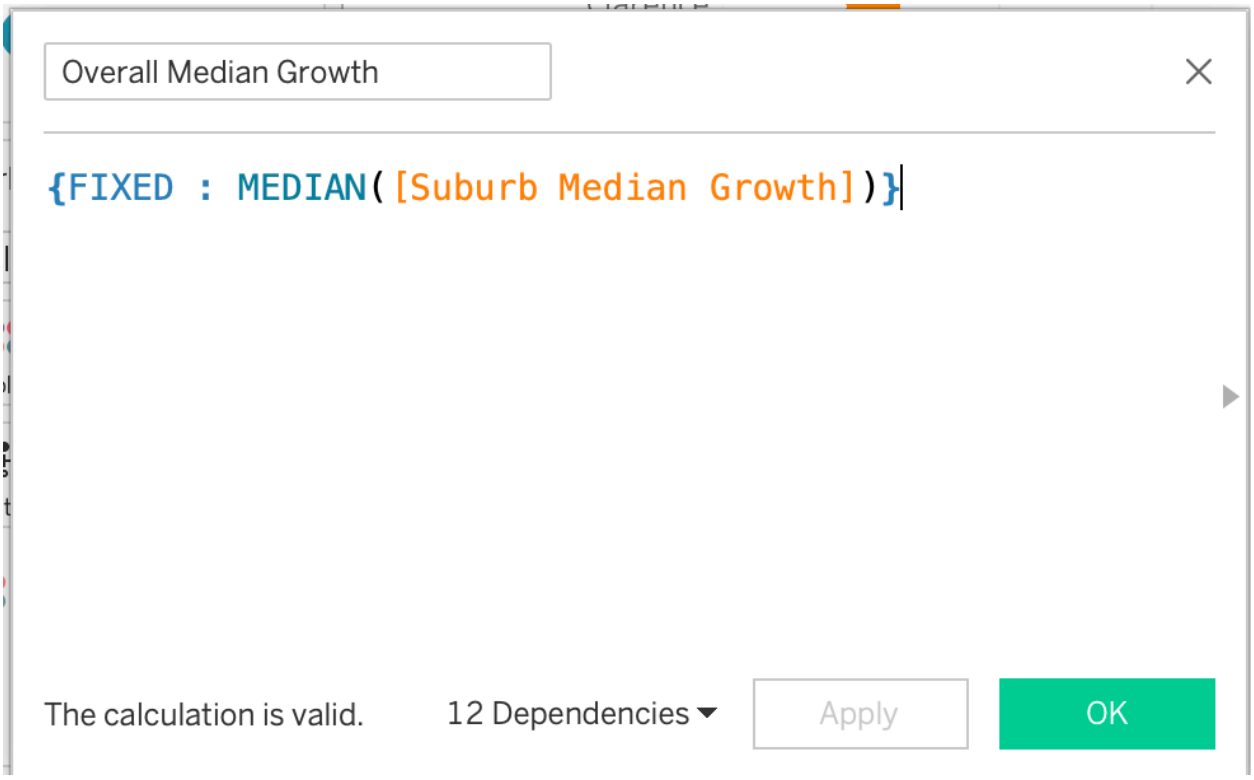
We need to do some setup in order to create this Suburb Ranking. First we create Suburb median growth using the Property Locality and getting the median of the Price Change %.



Now we create Suburb Median Price/m^2 using the Property Locality and the Median Price/m^2



In the same way we will create overall median growth and overall median price/m^2





Now we use this logic in order to segregate the values into separate quadrants:

IF [Suburb Median Growth] > [Overall Median Growth] AND [Suburb Median Price Per Sqm] < [Overall Median Price Per Sqm]

THEN "High Growth, Still Affordable"

ELSEIF [Suburb Median Growth] > [Overall Median Growth] AND [Suburb Median Price Per Sqm] >= [Overall Median Price Per Sqm]

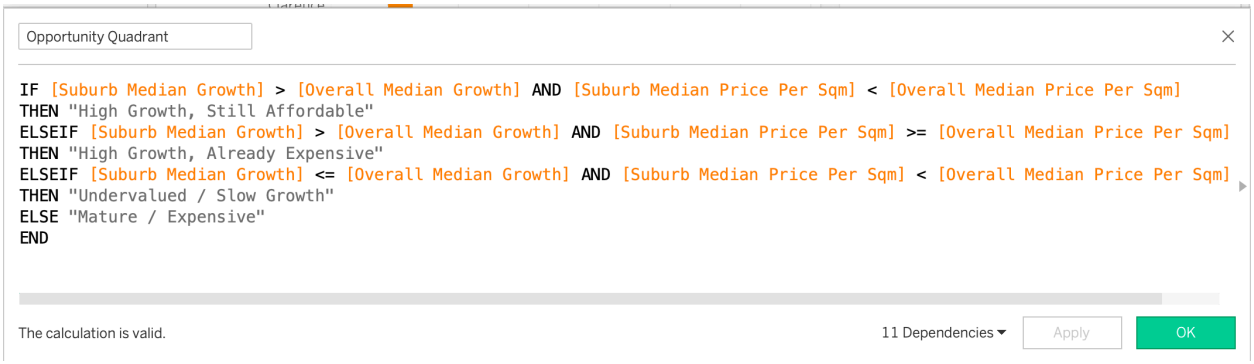
THEN "High Growth, Already Expensive"

ELSEIF [Suburb Median Growth] <= [Overall Median Growth] AND [Suburb Median Price Per Sqm] < [Overall Median Price Per Sqm]

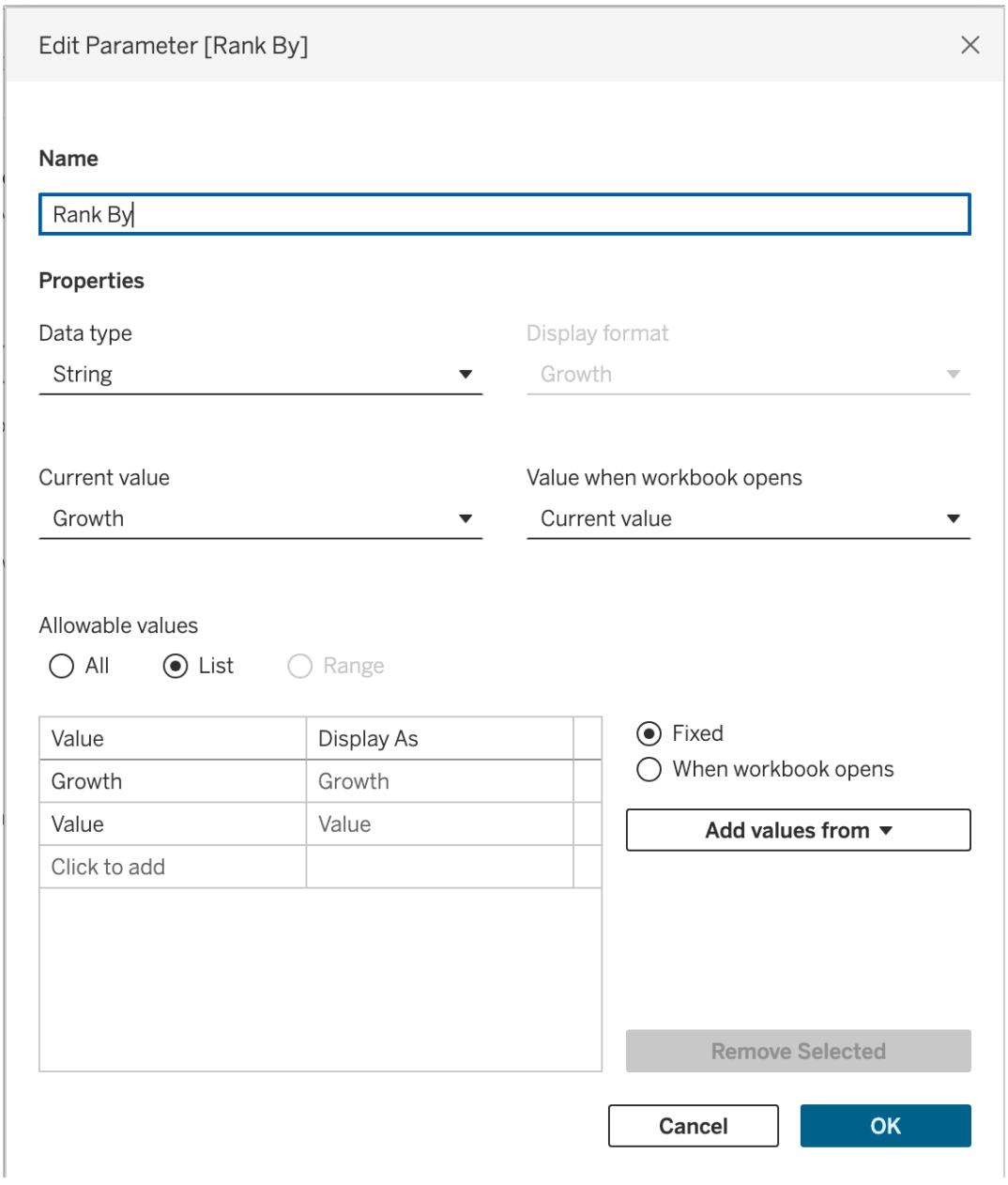
THEN "Undervalued / Slow Growth"

ELSE "Mature / Expensive"

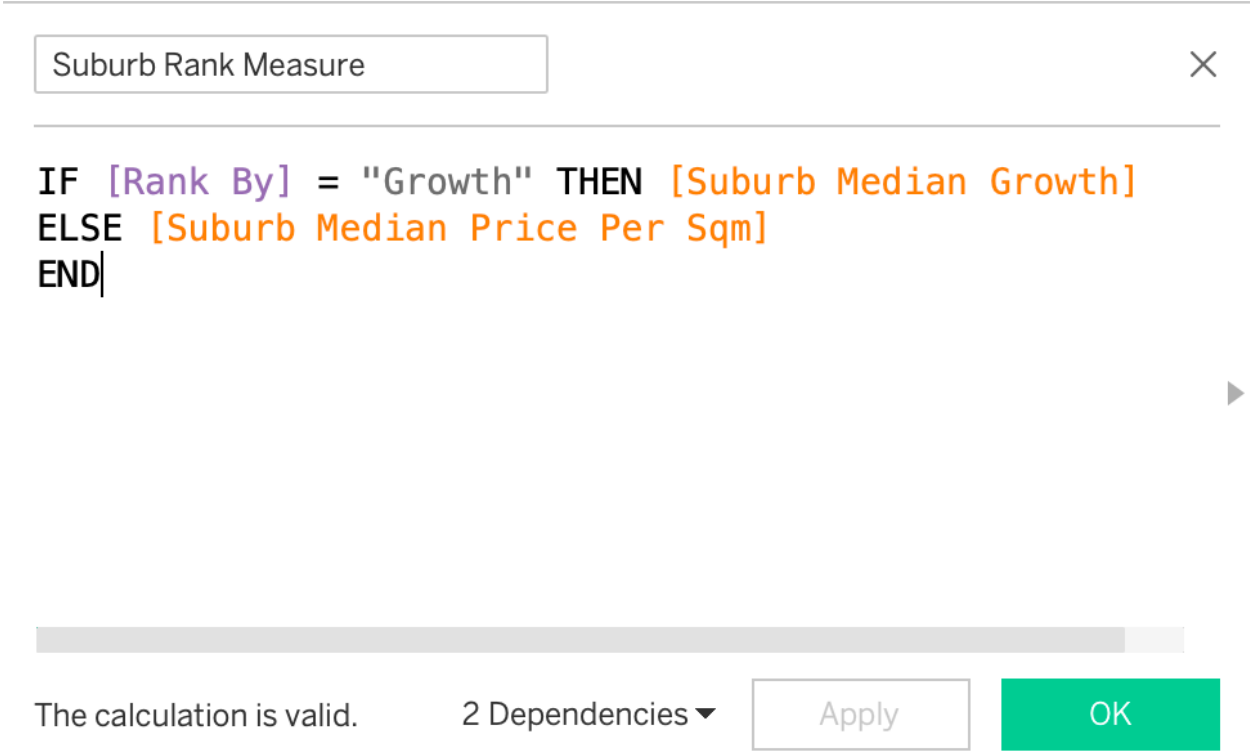
END



We also need to create a Parameter Rank By so we can sort the values by using Property Value or Property Growth.



Now to manipulate this Parameter we create another calculation that is Suburb Rank Measure which we will be using inside the Bar Chart.

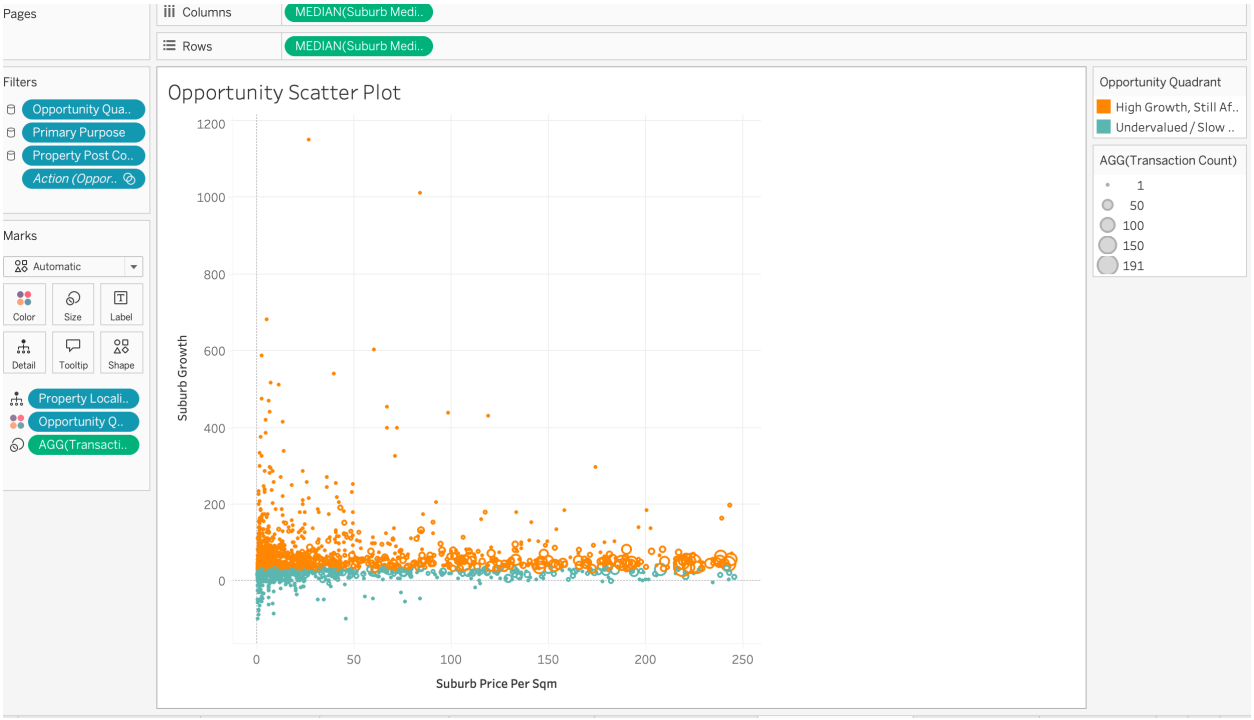


Now we just use this all together where we put column as Suburb Rank Measure, row as Property Locality. In the marks we show Opportunity Quadrant as colour and Median Previous Price/m^2 as detail. We shall also show the Parameter Rank By, Opportunity Quadrant and Post Code in filters so user can target their own preferred locations or Opportunity Quadrants.



Scatter Plot using Suburb Median Growth vs Suburb Median Price/m^2

Putting Suburb Median Price/m^2 as column and Suburb Median Growth as row having Opportunity Quadrant as colour, Property Locality as Detail and Transaction Count as size in marks we get our Scatter Plot which tell us how the localities lie in terms of growth and value.

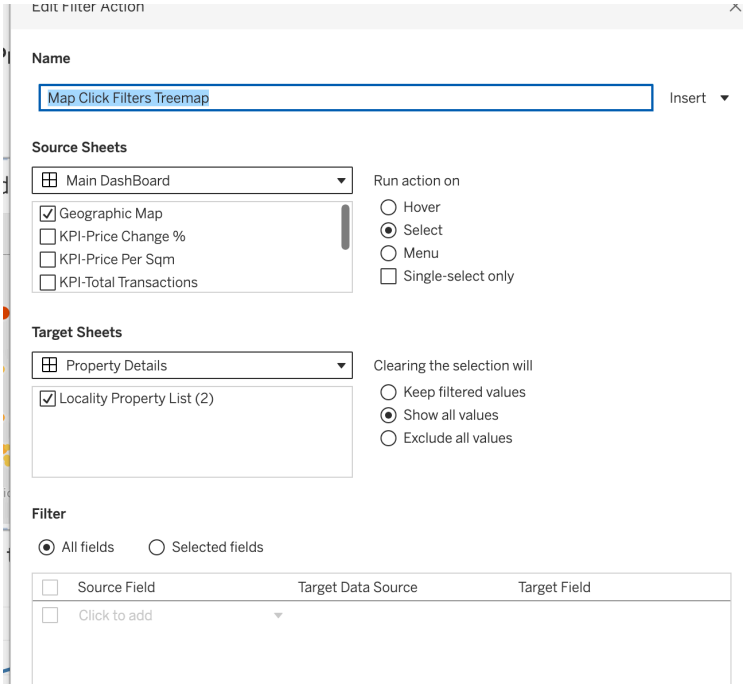


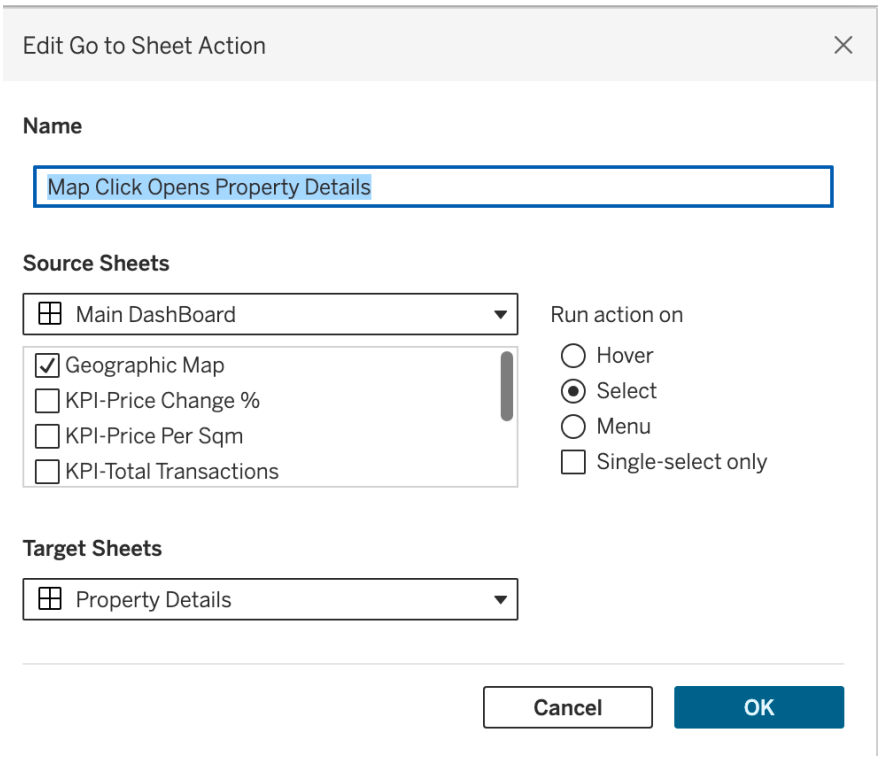
Locality Property List Treemap (Address-level drill-down with Opportunity Scores and tooltips)

Also since our clients need details for the address that he can follow up to and check details. We will create a drill down from the geographic map to the Locality Property Tree.

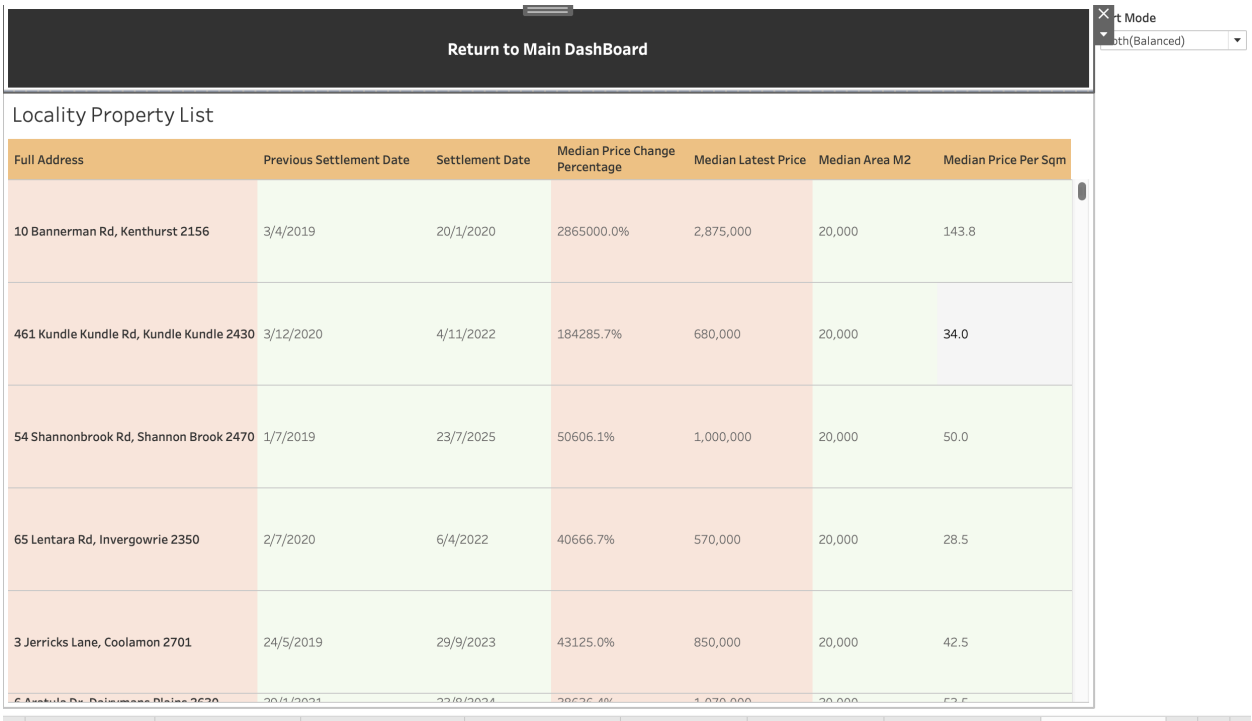
Go to Dashboard on the top menu and the Actions..., then we create this action which filters the data in the locality property list.

Also create a action that on click redirect the page to the new dashboard which has the property list.





Now create a table with Full Address, Price Change % , Latest Price, Area m^2, Price/m^2, Settlement Date and Latest Settlement Date. Also create a new Dashboard which we would be redirecting to when we click on the map.



6. Business & Strategic Recommendations

Systematic Quadrant 1 Target Acquisition ("High Growth, Still Affordable"): Prioritize capital deployment into postcodes categorized under Quadrant 1 in your Tableau model (suburbs where median capital growth exceeds the NSW state average, yet median price per square metre remains below state averages) to secure high upside prior to market repricing.
Outer-Ring Infrastructure Corridor Expansion: Focus acquisitions along expanding outer-ring corridors adjacent to major catalysts like the Western Sydney International Airport (Badgerys Creek corridor) before full operational activation.
Cash-Flow Hedging via High-Yield Niche Assets: Incorporate high-density commercial parking spaces into the portfolio mix to leverage their high price-per-square-metre yield as a cash-flow buffer against landholding costs.
Multi-LGA Risk-Adjusted Portfolio Hedging: Balance allocations across high-growth infrastructure corridors and stable, high-median Council Local Government Areas (LGAs) to hedge against localized market shifts.